Nile University

School of Information Technology and Computer Science

Program of Informatics

Row Pattern Matching Analytics: Bringing SQL MATCH_RECOGNIZE to Pandas DataFrames

A thesis submitted to the School of Information Technology and Computer Science at Nile University in Partial Fulfillment of the Requirements for the degree of Master of Science in Informatics.

By

Monier Ashraf Monier Lawande

Under Supervision of

Prof. Mohamed ElHelw

Prof. Ahmed Awad

***

Nile University

May, 2026

Thesis Supervision Agreement

Semester: Fall ☑ Winter ☐ Spring ☐ Summer ☐ Year: 2023 Date: 4/10/2023

Student Name: Monier Ashraf Monier Lawande

Student ID: 22100 2188 Program: Informatics

Thesis Topic: Row pattern matching analytics on PANDAS data frames.

Supervision Committee: 1. Prof. Mohamed El Helw

2. Prof. Ahmed Awad

Student Signature:	Date
Monier Ashraf Monier Lawande	4 OCT. 2023
Main Supervisor Signature:	**Date**
Mohamed A. ElHelw	8 Oct. '23
Supervisor Signature:	**Date**
Ahmed Awad	8/10/2023
Program Director Signature	**Date**
Sahar	8/10/2023

Copyright

This thesis was prepared and directed according to:

- Egyptian Standard No. 2609-2005 which is technically identical with International Standard No. ISO 7144:1986 for the preparation of scientific theses.
- The standardized reference citations are formulated according to the style of (IEEE Style)

تم إعداد وإخراج هذه الرسالة وفقآ ل :

- **المواصفة القياسية المصرية رقم 2609-2005 المتماثلة فنيآ مع المواصفة الدولية رقم ISO 7144:1986 الخاصة بإعداد الأطروحات العلمية**
- **تم صياغة الاستشهادات المرجعية المقتن وفقآ لأسلوب (IEEE Style)**

ABSTRACT

SQL:2016 introduced MATCH_RECOGNIZE to support row pattern recognition, allowing data analysts to detect ordered sequences of rows that match according to a user-defined pattern. This feature is useful in many domains such as financial monitoring, climate analysis, security logs, sensor analytics, and e-commerce, where the significant information frequently constitutes a sequence of events rather than an isolated record. Several database systems, including Oracle, Snowflake, Trino, and Apache Flink, now provide row-pattern functionality, but Pandas does not offer a native equivalent for in-memory Python analytics. As a result, analysts must often reproduce sequential patterns with shifted columns, manual loops, and custom state management, which makes the code longer, less readable, and more prone to errors. This thesis investigates how core SQL:2016 MATCH_RECOGNIZE-style behavior can be implemented directly over Pandas DataFrames. The proposed engine parses SQL-based MATCH_RECOGNIZE queries from a string, constructs a structured query representation, validates the supported row-pattern clauses, translates pattern expressions into finite automata, and executes the resulting matcher over ordered DataFrame partitions. The implementation supports the evaluated Feature R010-style scope, including PARTITION BY, ORDER BY, PATTERN, DEFINE, MEASURES,skip policies, output modes, quantifiers, alternation, grouping, anchors, exclusions, PERMUTE, and selected navigation and aggregate functions. It also uses controlled predicate evaluation instead of executing raw Python expressions. The system was evaluated through targeted correctness tests, comparisons with Trino and Oracle reference outputs, and performance experiments on samples from the Amazon UK Products 2023 catalog. The results show that the proposed approach preserves the evaluated row-pattern semantics while reducing developer effort compared with hand-written Pandas implementations. The thesis demonstrates that bringing MATCH_RECOGNIZE-style functionality

to Pandas can bridge SQL and Python workflows, making sequential data analysis more readable, portable, and productive.

Keywords: `MATCH_RECOGNIZE`; Row Pattern Recognition; Pandas; SQL:2016; Finite Automata; Pattern Matching.

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to my thesis advisor, Professor Ahmed Awad, for his constant academic, technical, and personal support throughout this work. His guidance, encouragement, and continued support at every moment meant more to me than words can express, and I am truly thankful. I am also sincerely grateful to Professor Mohamed ElHelw for his valuable guidance, steady support, and understanding, especially during difficult and unusual circumstances.My heartfelt thanks go to my family my mother, father, and brother for their endless love, patience, and support throughout all my years of study.I would also like to thank my friends and colleagues for their support and encouragement throughout this journey.

CONTENTS

Page

LIST OF TABLES

Table **Page**

LIST OF FIGURES

Figure **Page**

LIST OF ABBREVIATIONS

Full Name	Abbreviation
Full Definition	Shortcut
Full Definition	Shortcut
Full Definition	Shortcut

CHAPTER 1
INTRODUCTION

Pattern matching is the process of analyzing a sequence of data points whether rows in a relational table or events in a data stream with the goal of identifying specific shapes, trends, or behavior sequences within the data. It plays a crucial role across many analytical domains: detecting chains of transactions associated with money laundering in financial systems [1], revealing multi-step fraudulent behavior sequences in security event logs, identifying equipment degradation patterns in IoT sensor streams, and tracking multi-step user journeys in e-commerce clickstreams. Unlike conventional SQL predicates that evaluate each row in isolation, pattern matching takes the temporal ordering and relative values of successive rows into account. Patterns are described using a regular-expression-like syntax applied at the row level, enabling analysts to match and extract meaningful sequences across ordered partitions.

1.1 PROBLEM STATEMENT

The `MATCH_RECOGNIZE` (MR) clause was introduced in the SQL:2016 standard [2, 3], enabling users to define and search for patterns within ordered sequences of rows directly in SQL queries. Although standardized in 2016, only a limited number of database management systems have implemented it. Among established relational database systems, Oracle [4, 5] was the first to provide a production-quality implementation (in Database 12c, 2013), providing an early reference implementation for table-expression row pattern recognition. Several modern analytical platforms followed: Trino [6, 7], Snowflake [8], and Apache Flink [9] each provide their own implementations, though with varying degrees of conformance to the SQL:2016 feature levels (R010, R020, R030) and with important practical restrictions. Notably, PostgreSQL and DuckDB—two widely used open-source systems—do not yet support `MATCH_RECOGNIZE` [10, 11].

This gap is especially important for local analytical workflows. Many users prepare data, test hypotheses, and build downstream models directly in Python before moving to larger database or cloud systems. For data that fits comfortably in memory, requiring a separate SQL engine only to express row-pattern logic can add deployment and data-movement overhead. The focus of this thesis is therefore not to replace enterprise or streaming systems, but to bring a practical subset of declarative row-pattern recognition into the Pandas environment where many analytical workflows already begin.

Without this declarative clause, detecting multi-row patterns requires complex combinations of SQL window functions, self-joins, or procedural loops over rows outside the database. As the pattern grows beyond two or three steps, the corresponding code expands into dozens of lines filled with row-number calculations, timestamp comparisons, and cascading `JOIN` conditions. Such code is difficult to write, harder to maintain, and typically forces multiple table scans with poor performance [12].

In Python's data science ecosystem, the limitation is even more acute. Pandas [13] is a widely used library for in-process tabular data analysis, yet it provides no native sequential pattern primitive. Analysts who need to detect multi-step patterns must manually compose low-level operations: creating multiple shifted column copies (`df['value'].shift(-1)`, `shift(-2)`, ...), building Boolean masks, stitching subsequences with `groupby().apply()`, and pivoting or renaming columns. Listing 1.1 illustrates this approach for a simple three-step decreasing-price pattern (`A B C`).

```
import pandas as pd

df['order_date'] = pd.to_datetime(df['order_date'])
df = df.sort_values(['customer_id', 'order_date']).
   reset_index(drop=True)

def match_pattern_ABC(group: pd.DataFrame):
    matches = []
    for i in range(len(group) - 2):
```

```
        A, B, C = group.iloc[i], group.iloc[i+1], group.iloc[
            i+2]
        if (B['price'] < A['price']) and (C['price'] < B['
            price']):
            matches.append({
                'customer_id':  A['customer_id'],
                'start_date':   A['order_date'],
                'end_date':     C['order_date'],
                'bottom_price': C['price']
            })
    return matches

all_matches = []
for _, grp in df.groupby('customer_id'):
    all_matches.extend(match_pattern_ABC(grp))

result_df = pd.DataFrame(all_matches)
```

Listing 1.1: Detecting a three-step price-decrease pattern in Pandas without *MATCH_RECOGNIZE*

This hand-crafted approach is verbose (over 20 lines for a three-step pattern), prone to off-by-one alignment errors, and does not scale gracefully to longer or conditional patterns. The same query expressed with MATCH_RECOGNIZE is shown in Listing 1.2: a single declarative statement that specifies the partition, ordering, pattern grammar, variable definitions, and output measures.

```
import pandas as pd
from pandas_match_recognize import match_recognize

df['order_date'] = pd.to_datetime(df['order_date'])

query = """
SELECT customer_id, start_date, end_date, bottom_price
FROM orders
MATCH_RECOGNIZE (
  PARTITION BY customer_id
  ORDER BY     order_date
  MEASURES
    A.order_date AS start_date,
    C.order_date AS end_date,
```

```
    C.price       AS bottom_price
  ONE ROW PER MATCH
  AFTER MATCH SKIP TO NEXT ROW
  PATTERN (A B C)
  DEFINE
    B AS B.price < A.price,
    C AS C.price < B.price
)
"""

output_df = match_recognize(query, df)
```

Listing 1.2: The same pattern expressed with `MATCH_RECOGNIZE` on a Pandas DataFrame

The declarative form is not merely shorter syntactically: it separates what to find (the pattern) from how to find it (the engine), making patterns composable, auditable, and easy to extend. A more complex pattern, such as four or five steps with conditional alternation, requires no change to the surrounding Python code, only an update to the `PATTERN` and `DEFINE` clauses [14, 15]. It also allows the implementation to compile the row pattern into automata, cache repeated pattern structures, and process ordered partitions without exposing this execution logic to the user.

The full structure and sub-clauses of `MATCH_RECOGNIZE` are introduced in Chapter 2. Chapter 3 surveys the existing implementations and identifies the research gap.

1.2 RESEARCH OBJECTIVES AND CONTRIBUTIONS

This thesis addresses the gap between the SQL:2016 `MATCH_RECOGNIZE` standard and Python's Pandas ecosystem, bridging the expressiveness of relational pattern queries with the flexibility of in-memory analytics. Three objectives guided the work; each is stated below together with the contribution that fulfills it.

1. **An in-process engine supporting core SQL:2016 Feature R010-style capabilities for Pandas DataFrames.** Without a declarative alternative in Python, a data

scientist must write the twenty-line hand-crafted loop in Listing 1.1 merely to detect a simple three-step declining-price pattern. This thesis delivers an in-process `MATCH_RECOGNIZE`-style implementation supporting the core Feature R010 behavior evaluated in this thesis. It runs entirely alongside Pandas—no external database server or cloud subscription required—reducing that loop to a single declarative call in Listing 1.2 and eliminating the need for complex Python code or external SQL-engine dependencies.

2. **Broad evaluated coverage of SQL:2016-style pattern constructs.** The engine implements and evaluates a broad set of pattern constructs: quantifiers (`*`, `+`, `?`, `{n,m}`), alternation (`|`), grouping, anchors, exclusions (`{- -}`), and `PERMUTE`. For example, a fraud analyst can express a probe-and-drain sequence as

   ```
   PATTERN (PROBE+ (TRANSFER | WITHDRAW){2,5} DRAIN)
   ```

   and the engine parses, compiles, and executes it. Functional correctness is assessed through targeted automated tests covering the supported features. One acknowledged boundary: patterns with three or more levels of nested greedy quantifiers (e.g., `(A+B*)+C?`) can trigger exponential automata state-space growth; bounded quantifiers and simpler nesting behave efficiently [14].

3. **Safe predicate evaluation and empirical validation at a real-world scale.** User-written `DEFINE` predicates must be evaluated *securely*: passing strings directly to Python's `eval()` is a code-injection vulnerability (a predicate such as `__import__('os').system('echo hacked')` would execute an arbitrary system command). The engine instead parses predicates into an AST and evaluates them through a controlled interpreter that allows only permitted operations. Correctness is assessed through targeted automated tests, while performance is evaluated using the Amazon UK Products 2023 catalog [16]. The performance study uses five pattern families at five input sizes (25K, 35K, 50K, 75K, and

100K rows) sampled from the 2.2-million-row catalog; all 25 performance runs completed successfully, with throughput up to approximately **12 799 rows/sec** on a commodity laptop. Future work will address performance on larger datasets, memory consumption for high-variable patterns, and integration with query-optimizer plan-level improvements.

1.3 ORGANIZATION OF THE THESIS

The remainder of this thesis is organized as follows.

Chapter 2 — Background and Concepts presents the theoretical and technical foundations on which this work is built. It introduces sequential pattern matching and its real-world applications, explains the SQL:2016 `MATCH_RECOGNIZE` clause and its eight sub-clauses in detail, and introduces the SQL:2016 feature-level classification (R010, R020, R030) used throughout the thesis.

Chapter 3 — Related Work surveys each major `MATCH_RECOGNIZE` implementation in depth—Oracle, Trino, Snowflake, Apache Flink, and DuckDB—documenting their documented feature scope, execution strategies, deployment models, and practical restrictions. It also examines the Python and Pandas ecosystem gap and formally positions the contribution of this thesis within the broader pattern-matching research landscape.

Chapter 4 — System Design and Methodology describes the overall system architecture and methodology. It covers the SQL parsing pipeline, grammar and AST design, semantic validation, logical plan construction, and the design choices that connect declarative `MATCH_RECOGNIZE` queries to the execution layer.

Chapter 5 — Implementation and Optimization describes the concrete implementation of the engine. It covers finite-automata compilation, NFA and DFA construction, Pandas-based execution, output materialization, measure evaluation, and performance

optimizations such as pattern caching, memory management, and enhanced condition evaluation.

Chapter 6 — Experimental Evaluation presents the evaluation methodology, the benchmark dataset (Amazon UK Products 2023, 2.2 million rows), and experimental results covering functional correctness, performance across five pattern families and five input sizes, scalability, pattern complexity, and comparison with existing systems.

Chapter 7 — Conclusions and Future Work summarizes the main contributions, reflects on the limitations of the current implementation, and outlines directions for future research and further extensions.

1.4 PUBLICATIONS IN THE COURSE OF THIS THESIS

This thesis has resulted in scientific outputs contributing to the area of data systems and sequence-based pattern recognition. The following items summarize the published and submitted work produced during the course of this research.

First Publication. The primary outcome of this thesis has been published in the *2026 IEEE 23rd Mediterranean Electrotechnical Conference (MELECON)*. The paper, entitled "Row Pattern Recognition for Pandas: Bringing MATCH_RECOGNIZE to Python DataFrames" [17], addresses the absence of native row pattern recognition (RPR) support in Python data analysis frameworks. It proposes an SQL-in-Pandas engine that enables the execution of `MATCH_RECOGNIZE`-like queries directly on `pandas` DataFrames. The system supports key SQL:2016 RPR constructs and uses a non-deterministic finite automaton (NFA)-based matching approach for processing sequential patterns. Experimental results demonstrate reduced implementation complexity compared with conventional `pandas`-based workflows while maintaining correctness across the evaluated cases.

Second Publication. A second manuscript extending this work has been submitted to *Scientific Reports* and is currently under review.

1.5 CHAPTER SUMMARY

This chapter introduced the research problem addressed by the thesis: the absence of a native, declarative row-pattern recognition primitive in Pandas. It explained why hand-written Pandas solutions for multi-row patterns are verbose and error-prone, and it stated the main objectives and contributions of the work.

The chapter also outlined the structure of the thesis and listed the research outputs produced during the project. The next chapter provides the background needed to understand SQL row-pattern recognition and the `MATCH_RECOGNIZE` clause.

CHAPTER 2
BACKGROUND AND CONCEPTS

This chapter establishes the theoretical and practical foundations required to understand the design and implementation of the `MATCH_RECOGNIZE` engine for Pandas DataFrames. Section 2.1 introduces the problem of sequential pattern detection in relational data. Section 2.2 examines the SQL:2016 `MATCH_RECOGNIZE` clause and the terminology used throughout the thesis. The chapter closes with a worked V-pattern example that connects the standard concepts to the Pandas-based implementation. The next chapter then surveys related work and identifies the research gap addressed by this thesis.

2.1 PATTERN MATCHING IN SEQUENTIAL DATA ANALYSIS

Sequential pattern detection represents a fundamental challenge in data analysis, where the objective is to identify ordered sequences of events or states within large datasets. In contrast to conventional queries that match rows individually against one or more predicates, pattern matching takes preceding and subsequent rows into consideration and returns groups of rows when they follow a specified pattern. This makes it possible to retrieve specific shapes—such as V-, W-, or U-shapes—within the data. Pattern matching is therefore well suited to event-driven processes such as security monitoring, stock-price analysis, sensor-data analysis, fraud detection, and complex event processing [18].

Unlike traditional relational operations that focus on individual rows or aggregated values, pattern matching examines the temporal or logical ordering of data points to extract meaningful sequences. This capability is essential across many domains including financial market analysis, fraud detection, IoT sensor monitoring, and behavioral analytics.

The important distinction is that the "symbols" in row-pattern recognition are not characters in a string. They are data rows that become associated with pattern variables such as `A`, `B`, or `DOWN` when their predicates hold. A row sequence can therefore be interpreted as a sequence of variable assignments, which explains why regular-expression-like syntax and finite automata are natural tools for this problem. This connection is used later in the thesis when patterns are tokenized, compiled into NFAs, and executed over ordered Pandas partitions.

Historically, relational database systems excelled at set-based operations such as selection, projection, joins, and aggregation, but lacked expressive mechanisms for sequence analysis. Analysts requiring pattern detection were forced to export data to specialized stream-processing systems, implement procedural logic using cursors and temporary tables, or employ application-layer processing with significant performance overhead. These limitations created a gap between the widespread availability of sequential data and the tools available to analyze it effectively. The introduction of row pattern recognition in SQL:2016 addresses this deficiency by bringing sequence analysis capabilities directly into the query language [19].

2.1.1 Real-World Applications

Financial Market Analysis. Detecting V-shaped price patterns—price decline followed by recovery—requires examining ordered sequences of price movements. For example, identifying stocks whose price troughs are successively higher (a "higher-low" pattern) signals a potential upward trend. The canonical example used throughout this thesis is the detection of such V-patterns in customer order data [18].

Fraud Detection. Credit card fraud often manifests as specific transaction sequences, such as multiple small test charges followed by a large purchase, or geographically impossible transactions occurring within unrealistic timeframes. Expressing these rules declaratively in SQL is more maintainable than equivalent procedural logic [1].

IoT and Sensor Data. Manufacturing systems monitor sensor readings to detect equipment degradation patterns, such as a gradual temperature rise followed by a sudden drop, which may indicate imminent failure. Pattern queries over time-series sensor streams are a natural fit for `MATCH_RECOGNIZE` [18].

Clickstream and Funnel Analysis. E-commerce platforms analyze user navigation sequences to identify conversion paths or abandonment patterns. A classic funnel query detects the sequence "view → add-to-cart → remove → exit," which signals purchase hesitation. With `MATCH_RECOGNIZE`, the analyst can also quantify *how many* add-to-cart events occurred before removal, extract the exact timestamps of each step, and compute the total dwell time in a single declarative query—tasks that would otherwise require multiple correlated subqueries or procedural session-stitching logic. Such funnel-analysis workloads are common examples of row-pattern recognition in analytical SQL [20].

Network Security and Intrusion Detection. Cyber-attacks rarely manifest as a single anomalous event; they unfold as ordered sequences of actions. A typical multi-stage intrusion follows the pattern: port scanning → repeated failed authentication attempts → a single successful login → privilege escalation. `MATCH_RECOGNIZE` can express this entire kill-chain as one pattern query over an event log, flagging the offending session and extracting the first and last timestamps, the targeted port, and the account compromised—all without exporting the log to a separate SIEM tool [1].

Healthcare and Patient Monitoring. In clinical data streams, early warning systems are designed to detect physiological deterioration before a critical event. A typical sepsis-onset pattern consists of rising heart rate rows, followed by a drop in blood pressure rows, followed by a spike in white-blood-cell count. This example illustrates how ordered clinical measurements could be modeled as a row-pattern query over a

time-series of vital signs, while the exact alerting workflow would remain domain- and system-dependent.

Supply Chain and Logistics. Operational disruptions in supply chains often emerge as a cascade: a single late shipment → repeated partial deliveries → a stockout event. `MATCH_RECOGNIZE` applied to an order fulfillment log can identify such escalation sequences per supplier or warehouse and measure the time between first delay and eventual stockout, supporting earlier investigation rather than purely retrospective reporting.

Financial Compliance and Trade Surveillance. Trade-surveillance workloads often search for potential market manipulation patterns, for example "layering," where a trader places a large resting order (to move the price), then executes a trade on the opposite side, then cancels the resting order. This three-step sequence—place, trade, cancel—with precise timing constraints is naturally expressed as a `MATCH_RECOGNIZE` pattern, turning a complex compliance rule into an auditable SQL query.

2.2 THE SQL:2016 `MATCH_RECOGNIZE` CLAUSE

The SQL:2016 standard introduced `MATCH_RECOGNIZE` [19, 2], a construct that enables row pattern matching in SQL. Row pattern matching finds ordered sequences of rows whose variable assignments satisfy a regular-expression-like pattern. The expression is regular in its structure, but its variables are row predicates rather than literal characters. This allows users to describe complex event sequences in a declarative form [21]. In a theoretical sense, `MATCH_RECOGNIZE` does not add new relational expressiveness: many row-pattern queries can be rewritten using joins, window functions, recursive queries, or procedural code. Its practical contribution is instead a concise abstraction for sequence-oriented logic that would otherwise be verbose and difficult to maintain.

At the time of writing, MATCH_RECOGNIZE is still absent from several widely used data systems, including PostgreSQL [10] and DuckDB [11]. This thesis contributes to the broader goal of making declarative row-pattern matching available in environments that lack native support, with a particular focus on Pandas DataFrames and in-process Python analytics.

The SQL standard distinguishes between different feature levels for row pattern matching, which can be used to classify implementations [22]. **Feature R010** provides the most basic support and serves as a basis for further features; it includes row pattern recognition inside the FROM clause and basic aggregate functions (MIN, MAX, SUM, COUNT, AVG) inside the MEASURES clause. **Feature R020** extends the areas of application: MATCH_RECOGNIZE can be used within the WINDOW and OVER clauses to describe windows more precisely. **Feature R030** allows extended aggregate support inside the MATCH_RECOGNIZE clause [3, 14, 23].

This thesis uses the SQL:2016 terminology as the conceptual frame, but the implementation claims are intentionally scoped. The proposed engine targets core R010-style behavior in the FROM clause and evaluates selected row-pattern constructs that are relevant to in-memory Pandas analytics. Features outside this scope, including full window-clause row-pattern recognition and complete vendor-specific extensions, are treated as related work or future work rather than as requirements of the implemented system.

2.2.1 Clause Structure and Components

The clause operates on a rowset (table or derived table) and is commonly described through eight interconnected sub-clauses, some mandatory and some optional, as shown in Listing 2.1. The PATTERN and DEFINE clauses form the core mandatory part of row pattern recognition; the remaining clauses control partitioning, ordering, output, and overlap behavior.

```
1 SELECT <columns>
2 FROM   <table>
```

```
MATCH_RECOGNIZE (
    [PARTITION BY <partition_columns>] -- 1. Data
       organization
    [ORDER BY     <ordering_columns>]  -- 2. Row sequencing
    [MEASURES     <expressions>]       -- 3. Output
       computations
    [<rows_per_match>]                 -- 4. Result
       granularity
    [AFTER MATCH SKIP <option>]        -- 5. Overlap handling
    PATTERN ( <pattern_regex> )        -- 6. Sequence
       specification
    [SUBSET <union_variable_defs>]     -- 7. Union variables
    DEFINE <variable_definitions>      -- 8. Variable
       predicates
) AS <alias>
```

Listing 2.1: General syntax of the `MATCH_RECOGNIZE` clause

1. **PARTITION BY.** When supplied, divides the input rowset into independent partitions, analogous to window-function partitioning. Pattern matching runs independently within each partition, enabling both parallel processing and domain-specific grouping (e.g. per customer or per sensor). If omitted, the input is treated as a single partition.

2. **ORDER BY.** Establishes the logical ordering of rows within each partition. When supplied, the pattern-matching algorithm processes rows strictly according to this ordering; for deterministic sequence analysis, an explicit ordering is normally required. Multiple sort keys with `ASC`/`DESC` modifiers are supported.

3. **MEASURES.** Optionally defines computed expressions that extract values from matched row sequences using pattern variables and navigation functions (`FIRST`, `LAST`, `PREV`, `NEXT`). Each measure expression becomes a column in the output, allowing a match to be summarized without returning every raw input row.

4. **Output options.**

- `ONE ROW PER MATCH`: one summary row per complete match.
- `ALL ROWS PER MATCH`: one row for every row participating in a match.
- `WITH UNMATCHED ROWS`: an `ALL ROWS PER MATCH` modifier that includes non-matching rows in the result where supported.

5. **AFTER MATCH SKIP.** Controls how the engine resumes after finding a match:
   - `SKIP TO NEXT ROW` – resume from the row after match start (maximum overlaps).
   - `SKIP PAST LAST ROW` – resume from the row after match end (non-overlapping matches).
   - `SKIP TO FIRST` *var* – resume from the first row classified as *var*.
   - `SKIP TO LAST` *var* – resume from the last row classified as *var*.
6. **PATTERN.** Specifies the sequence using regular-expression-like syntax with pattern variables as terminals:
   - Concatenation: `A B C` (A followed by B followed by C).
   - Alternation: `A | B` (either A or B).
   - Quantifiers: `A*` (zero or more), `A+` (one or more), `A?` (zero or one), `A{n}` (exactly *n*), `A{n,m}` (between *n* and *m*).
   - Grouping: `(A B)+` (one or more AB pairs).
   - Anchors: `^A B C$` (pattern must span the entire partition).
   - Exclusions: `{- A+ B+ -}` (rows matching the enclosed sub-pattern are excluded from row-level output).
   - Permutations: `PERMUTE(A, B, C)` (any ordering of A, B, and C).
7. **SUBSET.** An optional clause for defining union variables, each of which consists of a set of variables already defined in the `PATTERN` clause. A union variable does

not introduce new matching behavior; it provides a shared name for rows matched by any of its component variables, and may appear inside `MEASURES`, `DEFINE`, and skip expressions where supported.

8. **DEFINE.** Assigns Boolean predicates to pattern variables. During matching, candidate rows are evaluated against these predicates in the current match context to determine whether they may be classified as a given variable. Predicates may reference current-row columns, navigation functions, aggregations over matched subsequences, and Boolean/comparison operators [22]. A pattern variable with no `DEFINE` predicate acts as an unconstrained variable.

A `MATCH_RECOGNIZE` clause is embedded in a `SELECT` query and can therefore be further processed by additional specifications. Possible extensions include applying groupings or aggregate functions to the outer `SELECT` [22].

Conceptually, these sub-clauses separate *what* pattern should be detected from *how* the engine searches for it. The `PATTERN` clause defines the shape of the sequence, the `DEFINE` clause defines the predicates that label rows, and the output and skip clauses determine how matches are materialized. This separation is central to the design of the implementation in later chapters: parsing produces a structured representation of the query, validation checks that the referenced variables and columns are consistent, and the pattern component is then compiled into an automaton for execution.

2.2.2 Illustrative Example: Stock Price V-Pattern

Listing 2.2 shows a `MATCH_RECOGNIZE` query that detects V-shaped price patterns in customer order data.

```
SELECT customer_id,
       start_price, bottom_price, final_price,
       start_date,  final_date
FROM   orders
MATCH_RECOGNIZE (
```

```
    PARTITION BY customer_id
    ORDER BY     order_date
    MEASURES
        START.price          AS start_price,
        LAST(DOWN.price)     AS bottom_price,
        LAST(UP.price)       AS final_price,
        START.order_date     AS start_date,
        LAST(UP.order_date)  AS final_date
    ONE ROW PER MATCH
    AFTER MATCH SKIP PAST LAST ROW
    PATTERN (START DOWN+ UP+)
    DEFINE
        DOWN AS price < PREV(price),
        UP   AS price > PREV(price)
);
```

Listing 2.2: Detecting V-shaped price patterns with `MATCH_RECOGNIZE`

The query partitions rows by `customer_id`, sorts by `order_date`, and seeks sequences matching `START DOWN+ UP+`: one `START` row (any row, as it carries no `DEFINE` predicate), one or more `DOWN` rows (each with a strictly lower price than its predecessor), and one or more `UP` rows (each with a strictly higher price than its predecessor). For every match found, the `MEASURES` sub-clause extracts the starting price and date from the `START` row, the lowest price from the last `DOWN` row, and the final price from the last `UP` row.

2.3 CHAPTER SUMMARY

This chapter introduced sequential pattern matching as an analytical problem and explained why ordinary row-by-row predicates are insufficient for many ordered-data tasks. It then described the main components of SQL:2016 `MATCH_RECOGNIZE`, including partitioning, ordering, measures, output modes, skip policies, pattern definitions, and variable predicates.

The stock-price V-pattern example showed how these components work together in a complete query and why the abstraction maps naturally to automata-based execution.

Chapter 3 examines existing implementations in greater depth, comparing their feature coverage, deployment models, and practical restrictions.

CHAPTER 3
RELATED WORK

This chapter positions the thesis within existing work on SQL row-pattern recognition and Python-based tabular analytics. Section 3.1 reviews systems that provide native `MATCH_RECOGNIZE` or related row-pattern support. Section 3.2 discusses the gap in Pandas and similar DataFrame libraries. Section 3.3 summarizes how this thesis addresses that gap.

Row pattern recognition was first proposed as a SQL standard feature by Zemke et al. [24] and was officially adopted in SQL:2016 [19, 2]. Subsequent editions refined and documented its semantics [3, 23, 14]. The clause has since been studied both as a specification and in terms of its practical implementations [15, 1].

The `MATCH_RECOGNIZE` clause is supported natively by several SQL and stream-processing engines, including Oracle [4, 5], Snowflake [8], and Trino [6, 7]; Apache Flink also exposes row pattern recognition through Flink SQL [9]. By contrast, widely used systems such as PostgreSQL and DuckDB do not currently expose a native `MATCH_RECOGNIZE` implementation [10, 11]. In systems without native support, users typically emulate row-pattern matching using window functions, recursive queries, user-defined aggregates, or external procedural layers [25, 26, 12]. These alternatives can express many of the same ideas, but they are usually more verbose and harder to maintain than a declarative row pattern clause.

3.1 SQL AND STREAM-PROCESSING ENGINES WITH NATIVE SUPPORT

Oracle [4, 5], Trino [6, 7], Snowflake [8], and Flink [9] all provide SQL-based row pattern recognition, but they differ in feature coverage, execution environment, and practical deployment cost. Although `MATCH_RECOGNIZE` was already defined as part of the SQL:2016 standard, it has only been implemented in a limited number of DBMSs to

date [15]. Given the construct's ability to express sequence logic that would otherwise require verbose combinations of joins, window functions, or procedural code, it remains useful to extend row-pattern support to more analytical environments. Oracle remains a major reference implementation for SQL row pattern matching [4, 27, 5].

3.1.1 Comparative Overview of Existing Implementations

The SQL standard defines three feature profiles for classifying row pattern recognition implementations [3, 14]. **Feature R010** covers row pattern recognition in the `FROM` clause together with basic aggregate functions; **Feature R020** additionally permits row pattern recognition inside `WINDOW`/`OVER` clauses; and **Feature R030** enables extended aggregate support within the clause. In practice, Feature R010 is the common baseline, while support beyond R010 varies across engines. Current Trino documentation, for example, also describes row pattern recognition inside window structures [6]. Feature R030 is not widely adopted in publicly documented implementations.

Table 3.1 summarizes the main systems considered in this chapter alongside the implementation developed in this thesis. The table uses the SQL feature levels where they map cleanly to public documentation; otherwise, it describes the documented implementation scope more cautiously. The comparison is therefore based on documented feature scope, deployment model, and research relevance, rather than on a claim that all systems expose identical SQL semantics.

3.1.2 Oracle

Oracle has been a pioneer in SQL `MATCH_RECOGNIZE`, having introduced row pattern recognition in Database 12c. It provides a mature implementation for enterprise-scale relational analytics and serves as a major reference point in discussions of SQL row pattern recognition [4, 27, 5]. Publicly documented usage focuses on the `MATCH_RECOGNIZE` clause in the table expression context; the window-pattern feature level (R020) is therefore outside the documented Oracle usage considered in this thesis. Oracle also imposes

Table 3.1: Comparison of `MATCH_RECOGNIZE` implementations

Engine	Documented scope	Open Source	Deployment	License
Oracle	`FROM`-clause row patterns	No	On-premise / Cloud	Commercial
Trino	`FROM` + window row patterns	Yes	Distributed	Apache 2.0
Snowflake	`FROM`-clause row patterns + extensions	No	Cloud-native	Commercial
Apache Flink	SQL subset / streaming CEP-backed	Yes	Streaming cluster	Apache 2.0
DuckDB	No native implementation; research translation	Yes	In-process	MIT
This work	**Core R010-style capabilities evaluated here**	**Yes**	**In-process Python**	**Open-source**

implementation-specific restrictions, including limitations on Boolean expressions in `MEASURES` and restrictions around aggregate syntax. Despite this technical maturity, Oracle's commercial deployment model and operational requirements make it less accessible for users seeking lightweight or open-source experimentation.

3.1.3 Trino

Trino is an open-source distributed query engine designed for parallel, federated query execution across large data sources [6, 7]. Its current documentation covers `MATCH_RECOGNIZE` in the `FROM` clause and also documents row pattern recognition in window structures. This makes Trino one of the broader open-source implementations from a SQL-feature perspective, but it is still a distributed query engine rather than an in-process DataFrame library. Using it therefore requires server or cluster deployment and operational management that may be unnecessary for smaller Pandas workflows. In this thesis, Trino is useful primarily as a semantic reference point for validating selected query results, not as a substitute for a local DataFrame-native execution path.

3.1.4 Snowflake

Snowflake is a cloud-native data platform. It provides documented `MATCH_RECOGNIZE` support with rich `DEFINE` and `MEASURES` expression capabilities for row-pattern queries [8,

20]. Its documentation also notes implementation-specific behavior: the engine uses backtracking for pattern matching and warns that some pattern/data combinations can take a long time to execute. Some row-pattern functions are also restricted in particular sub-clauses. Because Snowflake is a managed cloud warehouse, computational cost scales with warehouse size and query workload.

3.1.5 Apache Flink

Apache Flink [28, 29] is designed for real-time stream processing and exposes `MATCH_RECOGNIZE` through Flink SQL, backed by its Complex Event Processing (CEP) runtime library [9]. Its documented implementation is a SQL subset oriented toward streaming use cases. The current Flink documentation explicitly describes the implementation as a subset of the full standard and lists unsupported features including pattern groups, some alternations, `PERMUTE`, anchors, exclusions, `ALL ROWS PER MATCH`, `SUBSET`, physical offsets (`PREV/NEXT`), and `DISTINCT` aggregations. While Flink is highly scalable and effective for high-velocity data, it assumes a streaming runtime and introduces its own learning curve for users accustomed to lightweight, batch-oriented analysis. For analytical workflows that combine pattern extraction with machine learning, these systems may also require additional integration work between SQL execution and Python-oriented modeling tools [30, 31]. In many practical pipelines, this can introduce data movement, duplicated intermediate data, or more complicated orchestration steps [32].

3.1.6 Bringing `MATCH_RECOGNIZE` to DuckDB

DuckDB is a relational database management system that offers several advantages over the DBMSs discussed above [33]. The most significant advantage is that it does not require a separate server for operation; instead it can be embedded directly into the host application, making it well-suited for small projects and in-process analytics. DuckDB is open-source, enabling users to contribute new functionality. Its Python API also makes it convenient to test and debug generated SQL from translation-based approaches [33].

Although MATCH_RECOGNIZE has not yet been implemented in DuckDB natively, DuckDB supports relevant SQL building blocks such as window functions and recursive common table expressions. This makes it a natural target for research that transpiles MATCH_RECOGNIZE queries into standard relational constructs [11, 34, 25, 26]. Such approaches typically use multiple CTEs and rule-based rewriting to simulate pattern variables, ordering, skip behavior, and output measures. A DuckDB-oriented translation approach would therefore aim to cover core Feature R010 use cases without requiring a native execution operator inside DuckDB itself.

3.1.7 Implementation Restrictions Across Systems

Existing implementations of row pattern recognition impose implementation-specific restrictions during use. Examples reported across systems [4, 27, 6, 9, 8, 15] include:

- supported contexts differ, with some systems documenting row patterns only in the FROM clause and others also documenting window-oriented row pattern recognition;
- the availability of pattern-recognition functions such as CLASSIFIER() and MATCH_NUMBER() differs between MEASURES, DEFINE, and output modes;
- aggregate support differs across engines, including restrictions on DISTINCT aggregates and on expressions used as aggregate arguments;
- support for empty matches, exclusions, reluctant quantifiers, and skip modes is not uniform across all systems.

For many users—especially those with modest analytics needs or who work independently—installing and managing large-scale systems like Oracle, Trino, Snowflake, or Flink can be overly complicated. These users may only need occasional pattern detection and may prefer to avoid the overhead of learning different functions or dealing with varying syntax

for similar queries across platforms [35]. Table 3.1 summarizes the key characteristics and documented scope of the systems discussed above.

Additionally, for datasets that already fit comfortably in memory, Pandas can offer a lower-friction workflow than deploying a separate database server, cluster, or cloud warehouse [36, 37]. This does not replace the strengths of Oracle, Trino, Snowflake, or Flink for large-scale or streaming workloads; rather, it highlights a different use case: lightweight, local, Python-centered analysis where data preparation, pattern extraction, and downstream modeling can remain in the same environment.

3.1.8 Synthesis of the Research Gap

The reviewed systems show that row-pattern recognition is a mature and useful abstraction, but they also reveal a deployment and workflow gap. Oracle and Snowflake provide powerful managed or commercial database environments; Trino provides a distributed open-source SQL engine; and Flink targets streaming workloads. These systems are appropriate when data already resides in their execution environment or when the workload requires distributed processing. They are less convenient when the analyst is working with a Pandas DataFrame that already fits in memory and wants to remain inside a Python notebook, script, or experimental pipeline.

The research problem addressed in this thesis is therefore not the absence of `MATCH_RECOGNIZE` in SQL engines in general. Rather, it is the absence of a Pandas-native row-pattern interface that preserves the main declarative structure of SQL row-pattern recognition while avoiding a round trip through an external database or stream-processing system. This distinction defines the scope of the contribution: the proposed engine targets in-process, offline DataFrame analytics and evaluates core R010-style behavior and selected SQL:2016 pattern constructs. It is not intended to replace distributed engines for large-scale or streaming workloads.

3.2 PANDAS AND THE PYTHON ECOSYSTEM GAP

Pandas is a popular, open-source Python library for data analysis and manipulation [13]. Its straightforward installation process, broad community adoption, and integration with the Python data-science ecosystem make it attractive for students, researchers, and practitioners who need to analyze tabular data without deploying a separate database system. Pandas provides useful primitives for sorting, filtering, grouping, rolling windows, shifting columns, and basic time-series analysis.

These primitives are powerful, but they are low-level building blocks rather than a declarative row-pattern language. Expressing a multi-step sequence usually requires combinations of `groupby`, `shift`, `rolling`, custom Python loops, or intermediate Boolean columns. Such implementations become verbose and error-prone as patterns grow longer, require overlap control, or involve conditions over previously matched rows.

Despite this broad adoption, Pandas does not provide a native SQL:2016 `MATCH_RECOGNIZE`-style primitive for row pattern recognition. In this setting, analysts often need to manually implement stateful logic using `DataFrame.apply()` chains, `iterrows()`, or external database round-trips—all of which are error-prone, verbose, and difficult to maintain. Within the scope of this thesis, alternative DataFrame libraries such as Dask, Polars, and Modin similarly lack a direct equivalent to SQL row pattern recognition. Even DuckDB, which supports window functions and in-process analytics [11], does not yet expose a native `MATCH_RECOGNIZE` interface.

3.3 CONTRIBUTION OF THIS WORK

This thesis addresses the identified gap with the following contributions:

1. A **Pandas-native** `MATCH_RECOGNIZE` **engine** supporting core SQL:2016 Feature R010-style capabilities in an in-process Python workflow, without requiring a separate database server or cloud warehouse.

2. **Broad pattern construct support**, including quantifiers, alternation, grouping, anchors, exclusions, and `PERMUTE`, following the main row-pattern constructs used by SQL:2016 `MATCH_RECOGNIZE`.

3. An **optimization layer** comprising LRU pattern caching, partition-wise memory management, and thread-safe access to shared cache components.

4. A **safe expression evaluation pipeline** that reduces code-injection risk by parsing user-defined predicates into a controlled representation rather than executing raw Python expressions.

5. **Empirical validation** using targeted functional tests and experiments on samples from the Amazon UK Products 2023 catalog, with the detailed methodology and results reported in Chapter 6.

3.4 CHAPTER SUMMARY

This chapter reviewed the main systems that currently support, or partially support, SQL row-pattern recognition. Oracle, Trino, Snowflake, and Apache Flink provide useful reference points, while DuckDB and Pandas illustrate the current gap for lightweight in-process analytics.

The chapter showed that existing alternatives either require a separate database or stream-processing engine, or force analysts to write procedural logic in Python. The resulting gap is a scoped but important one: declarative row-pattern recognition for in-memory Pandas workflows. This motivates the system design presented in Chapter 4.

CHAPTER 4
SYSTEM DESIGN AND METHODOLOGY

The MATCH_RECOGNIZE implementation follows a modular, five-stage architecture specifically designed for SQL row-pattern recognition. The engine executes SQL queries directly on pandas DataFrames [38] while preserving the supported SQL:2016 row-pattern semantics [2, 3, 23]. The pipeline consists of five phases: *parsing*, *semantic validation*, *logical plan construction*, *pattern compilation*, and *execution* (Fig. 4.1).

In the parsing stage, the SQL query is transformed into an abstract syntax tree (AST) that captures both the SELECT/FROM clauses and MATCH_RECOGNIZE sub-clauses [39, 40]. Semantic validation then checks variable usage, predicates, skip policies, output modes, and the consistency of referenced row-pattern constructs. The validation rules are scoped to the features supported by the implementation: they enforce correct use of navigation functions, consistent measure semantics, and predictable handling of edge cases such as empty matches, partition boundaries, and skip behavior.

The validated plan is translated into a logical plan composed of modular operators for partitioning, sorting, matching, measuring, and skipping. From this plan, the system compiles a nondeterministic finite automaton (NFA) with guarded transitions that support quantifiers, alternation, and PERMUTE constructs [41]. The NFA is constructed using Thompson's construction, extended for SQL:2016 semantics by embedding predicates from the DEFINE clause when present, organizing ε-transitions, and applying subset construction with state deduplication to mitigate unnecessary state growth in practical patterns.

Finally, execution is performed over ordered partitions using the compiled automaton. Matches are materialized with the correct RUNNING/FINAL semantics. The evaluation engine manages context separation, applies navigation functions (FIRST, LAST, PREV, NEXT) with partition awareness, and computes aggregates such as COUNT, SUM, and

AVG across pattern variables. This preserves the supported SQL:2016 behavior while producing results directly as pandas DataFrames.

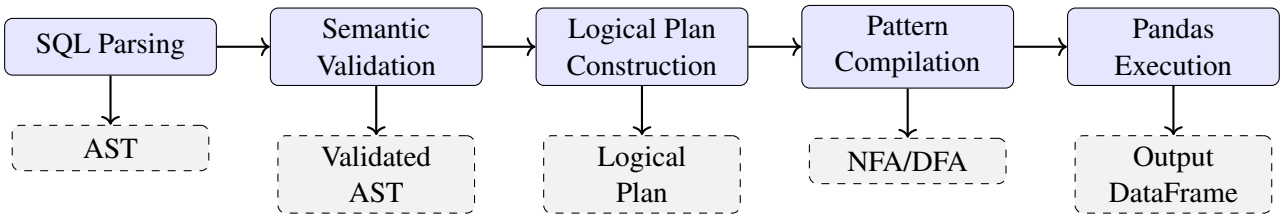

Figure 4.1: Five-stage architecture of the Pandas MATCH_RECOGNIZE engine

The remainder of this chapter is organized as follows. Section 4.1 provides a high-level overview of the three primary runtime components. Section 4.2 describes the SQL parsing pipeline in detail. Section 4.3 covers the semantic validation checks applied after parsing. Section 4.4 presents the logical plan construction step. The following chapter continues from this logical design into automata compilation, Pandas execution, implementation details, and optimization techniques.

4.1 SYSTEM ARCHITECTURE OVERVIEW

The engine's five-phase pipeline is designed so that each stage produces a well-defined artifact consumed by the next. The following three subsections give a brief description of the primary runtime components. This chapter then details the parsing, validation, and logical-planning stages; Chapter 5 continues with automata compilation, Pandas execution, and optimization.

4.1.1 SQL Parsing and AST Construction

The process starts by analyzing the user's SQL query and identifying its logical components. A parsing process explores the query, recognizes its main components—grouping (PARTITION BY), ordering (ORDER BY), the pattern to be searched, and any specific conditions or output metrics—and organizes these into an Abstract Syntax Tree

(AST). The AST provides a clear, structured representation of the query. This approach eases query validation and prepares it for pattern matching in the following stages.

4.1.2 Pattern Translation to Finite Automata

Once the AST is complete, the system focuses on the pattern component of the SQL query. The pattern is converted into a finite automaton that defines a set of states and transitions for detecting complex sequences or behaviors in the data. The process begins by constructing a non-deterministic finite automaton (NFA) from the pattern and then, for improved efficiency, transforms it into a deterministic finite automaton (DFA) [29]. The DFA eliminates transition ambiguity, enabling the system to scan data sequences quickly and accurately. This translation handles a wide variety of pattern features—repetitions, alternatives, and specific sequences—in a unified and efficient way. The resulting automaton acts as the core mechanism for scanning and detecting patterns within the data.

4.1.3 Execution Engine on Pandas

The execution phase uses the compiled DFA to perform pattern matching directly on input data represented as Pandas DataFrames. The engine begins by partitioning and ordering the data according to the query's specifications. It then applies the DFA to scan the dataset, identifying matching sequences according to the configured pattern and skip policy. For every detected match the engine computes any requested measures or calculations and compiles the results into a DataFrame formatted to mirror standard SQL query outputs. By utilizing Pandas, the system retains both flexibility and accessibility, smoothly incorporating advanced pattern matching into conventional Python data-analysis workflows.

4.2 SQL PARSING

Transforming a `MATCH_RECOGNIZE` query from plain text into an executable representation requires a parsing pipeline that handles both standard SQL syntax and the extended row-pattern sub-language. This section explains parsing fundamentals, surveys existing SQL parsers and parser generators, motivates the design choices made in this implementation, and describes the resulting parser in detail.

4.2.1 What is Parsing?

Parsing refers to the analysis of an input string according to the rules of a formal grammar and its transformation into a machine-processable representation [34]. In essence, the parser examines a human-written query, determines which grammar rules govern its syntactic construction, and produces a tree data structure that reflects that derivation.

Parsing in this engine proceeds in two sequential stages: *lexical analysis* and *syntactic analysis* [42].

Lexical analysis. The raw character stream is segmented into a sequence of predefined, meaningful units called *tokens*. Keywords (`SELECT`, `MATCH_RECOGNIZE`, `PATTERN`, ...), identifiers, string and numeric literals, and punctuation are all examples of tokens. Whitespace and comments are discarded at this stage; only the ordered token sequence is passed to the next stage.

Syntactic analysis. The token stream is matched against the grammar rules of the language, and a valid input is translated into a tree representation. A valid input produces a *parse tree* (also called a *concrete syntax tree*) whose internal nodes correspond to grammar rules and whose leaves correspond to individual tokens. An invalid input causes the parser to report an error, indicating precisely where the text violated the grammar.

The two-stage design separates concerns cleanly: the lexer handles regular-language aspects (token shapes), while the parser handles context-free-language aspects (phrase structure). Together they convert a character string into a structured tree that subsequent compiler phases can traverse and transform.

4.2.2 Parsing SQL: Existing Solutions

Several Python libraries offer SQL parsing out of the box. `sqlparse` [43] performs tokenization and basic structural splitting (statements, clauses) but produces only a shallow token stream, not a typed AST, and does not model `MATCH_RECOGNIZE` at all. `sqlglot` [44] is a more complete SQL transpiler that supports multiple SQL dialects and exposes a typed AST; however, it does not provide project-ready coverage for the required SQL:2016 row-pattern sub-language (e.g. exclusions, anchors, `PERMUTE`, bounded quantifiers). `mo_sql_parsing` [45] converts SQL to JSON but similarly provides no project-ready coverage for row-pattern constructs.

A widely used baseline for conventional SQL parsing is the **PostgreSQL parser**, which has been extracted from the PostgreSQL source code and released as a standalone C library via the `libpg_query` project [46]. Built on top of `libpg_query`, the Python library `pglast` [47] wraps the original C AST nodes as typed Python classes and provides functionality to reconstruct SQL text from an AST. Listing 4.1 shows a minimal usage example: the call to `parse_sql` returns a tree of typed node objects, and `RawStream` serializes it back to SQL.

```
from pglast import parse_sql
from pglast.stream import RawStream

tree_root = parse_sql("SELECT 42;")
print(tree_root)           # typed AST node objects
print(RawStream()(tree_root))  # SELECT 42
```

Listing 4.1: Parsing a SQL statement with `pglast` and reconstructing SQL text

Although `pglast` cannot parse `MATCH_RECOGNIZE`, it illustrates the difference between a conventional SQL AST and the row-pattern-specific structure required here. The project therefore does not depend on `pglast`, `sqlparse`, `sqlglot`, or `mo_sql_parsing` for execution. Instead, these tools were considered as alternatives, and a grammar-based parser was selected because the required syntax includes pattern constructs that are not handled uniformly by general-purpose Python SQL parsers.

4.2.3 Grammars

A grammar formally describes the syntax of a language [42]. The elementary constituents of a grammar are:

- **Terminal symbols** (tokens) — the elementary, indivisible symbols of the language. In this thesis the terms *terminal* and *token* are used interchangeably.
- **Non-terminals** — symbols that represent sets of strings composed of terminals, used to name syntactic constructs (e.g. *expression*, *pattern*, *clause*).
- **Production rules** — rewriting rules of the form $N \to \alpha$, where N is a non-terminal and α is a sequence of terminals and/or non-terminals. Each rule describes one way in which a particular construct of the language may be written [48].

A grammar is specified as a list of production rules. Because the left-hand side of each rule contains exactly one non-terminal, the grammar is *context-free*: the applicable rules do not depend on the surrounding context [42]. For example, a grammar for one-or-more occurrences of the letter a can be written:

$$S \to a \quad | \quad aS$$

This form is also known as *Backus–Naur Form* (BNF). The *Extended* variant (EBNF) adds syntactic sugar for optional elements (`[ ]`), repetition (`{ }`), and grouping (`( )`),

making large grammars more readable. For example, the `PARTITION BY` clause can be expressed in EBNF as:

```
partitionBy : 'PARTITION' 'BY' expr (',' expr)* ;
```

SQL is a large context-free language; the `MATCH_RECOGNIZE` sub-language adds additional syntactic constructs for pattern terms, quantifiers, alternation, grouping, anchors, exclusions, `PERMUTE`, and the eight sub-clauses.

4.2.4 Parsing Requirements

This section examines what must be parsed and what the output should look like. Listing 4.2 shows the supported syntax of a query containing a `MATCH_RECOGNIZE` clause in this implementation.

```
SELECT <expression> [, ...]
FROM   <table>
MATCH_RECOGNIZE (
  [ PARTITION BY <column> [, ...] ]
  [ ORDER BY <column> [ASC|DESC] [, ...] ]
  [ MEASURES <expression> AS <alias> [, ...] ]
  [ ONE ROW PER MATCH
  | ALL ROWS PER MATCH
    [ SHOW EMPTY MATCHES
    | OMIT EMPTY MATCHES
    | WITH UNMATCHED ROWS ] ]
  [ AFTER MATCH SKIP
    { PAST LAST ROW
    | TO NEXT ROW
    | TO FIRST <variable>
    | TO LAST  <variable> } ]
  PATTERN ( <row_pattern> )
  [ SUBSET <subset_name> = ( <variable> [, ...] ) [, ...] ]
  [ DEFINE <variable> AS <condition> [, ...] ]
) [ AS <alias> ]
<rest>;
```

Listing 4.2: Syntax of a `MATCH_RECOGNIZE` query

The `DEFINE` clause is optional in both the grammar and the execution pipeline. If a pattern variable appears in `PATTERN` without a corresponding predicate in `DEFINE`,

the logical plan assigns that variable an implicit always-true predicate. Measure aliases require the AS keyword in the adopted Trino grammar, while the alias after the MATCH_RECOGNIZE clause itself is optional.

MATCH_RECOGNIZE is an optional sub-clause of the FROM clause. The implementation focuses on one MATCH_RECOGNIZE clause extracted from a top-level SELECT query and supports an optional outer ORDER BY. More complex SQL compositions such as full CTE rewriting, set-operation planning, and general nested-subquery execution are outside the main execution path, although selected subquery-like and multi-query scenarios are covered by the test suite as foundations for future extension. This scope is sufficient for the core FROM-clause row-pattern use cases evaluated in this thesis.

The parser must extract the mandatory pattern clause and the available optional sub-clauses of MATCH_RECOGNIZE—PARTITION BY, ORDER BY, MEASURES, rows-per-match, after-match-skip, PATTERN, SUBSET, and DEFINE—and deliver them to subsequent pipeline stages through a well-defined AST interface, described next.

4.2.4.1 The MATCH_RECOGNIZE AST Interface

The AST exposed by the parser to downstream stages is defined by a hierarchy of typed Python @dataclass objects. The root node, FullQueryAST, holds the outer SelectClause, FromClause, and a single MatchRecognizeClause object that groups the parsed row-pattern sub-clauses. The interface contract is:

- Each node is represented as a typed Python dataclass or structured object. Some metadata fields, especially those on PatternClause, are normalized after construction so that defined variables, implicit variables, and subset components can be resolved consistently.
- Each MATCH_RECOGNIZE sub-clause maps to a dedicated typed dataclass (PartitionByClause, OrderByClause, MeasuresClause, RowsPerMatchClause, AfterMatchSkipClause,

`PatternClause`, `SubsetClause`, `DefineClause`), enabling the NFA compiler to dispatch on type rather than parsing raw strings.

- `DEFINE` predicates and `MEASURES` expressions are captured as expression strings during the first pass and interpreted on demand through a controlled expression evaluator (Section 4.2.12), keeping the main grammar concise.

- `Measure` nodes carry computed flags (`is_classifier`, `is_match_number`) that allow the execution layer to route output columns without additional string inspection.

4.2.5 Parser Generators

Since productivity is a goal, the question arises whether the creation of a lexer and parser can be automated. The kind of software addressing this issue is called a *parser generator* [34]: it takes a grammar as input and produces the source code of a parser that understands the syntax defined by that grammar. The target language of the generated source code must be Python, since that is the language the engine is written in.

Candidates that support Python as a target language include:

- **PLY** (Python Lex-Yacc) — a Python implementation of the classic 1970 Lex/Yacc tools [49].

- **Lark** — described as "a modern approach, with a focus on ergonomics, performance and modularity" [50].

- **ANTLR4** — a parser generator "widely used to build languages, tools, and frameworks; from a grammar, ANTLR generates a parser that can build and walk parse trees" [51].

Table 4.1 compares the three tools across criteria extracted from their respective documentation.

Table 4.1: Comparison of Python-compatible parser generators

Feature	**PLY (Lex/Yacc)**	**Lark**	**ANTLR4**
Algorithm	LALR(1)	LALR / Earley	Adaptive LL(*) [52]
Grammar notation	BNF	EBNF	EBNF
Ambiguity handling	Operator precedence declarations	Rule priorities; Earley resolves via priority	Production order & semantic predicates
Left recursion	Yes	Yes (LALR mode)	Yes
Target language	Python native	Python native	Java runtime required to generate Python parser
AST / parse tree	No	Yes (parse tree)	Yes (parse tree)
Tree visualiser	No	Dot output	Java GUI (`antlr4-tools`)
Traversal scaffolding	No	Transformer class; methods written manually	Listener & visitor generated automatically

Due to the manageable size and complexity of the `MATCH_RECOGNIZE` statement, any of the three tools would likely produce a satisfactory result. Nevertheless, two reasons motivated the choice of ANTLR4. First, its parse-tree visualiser (via the `antlr4-tools` Java GUI) greatly aids debugging grammar rules. Second, and more crucially, ANTLR4 automatically generates a traversal skeleton—both listener and visitor base classes—for the parse tree. Because the raw parse tree must be transformed into the typed AST described in Section 4.2.4.1, a generated visitor base class eliminates manual tree-traversal boilerplate. An empirical study further demonstrates significantly higher performance and maintainability when using ANTLR compared to Lex/Yacc [53].

In this thesis the workflow combining ANTLR4 with the typed `MATCH_RECOGNIZE` AST interface is evaluated for its suitability in parsing the supported `MATCH_RECOGNIZE` statement.

4.2.6 Alternative Methods

Two non-generator approaches were considered and rejected.

Hand-written recursive descent. A recursive descent parser is straightforward to implement for small grammars: each non-terminal becomes a Python method that calls other methods for sub-rules. However, SQL's grammar contains hundreds of productions; maintaining a hand-written parser as the grammar evolves is error-prone. The approach was ruled out on maintainability grounds.

Regex-based string extraction. Regular expressions cannot model recursive structures such as nested grouping `((A+B)*|C)` or balanced parentheses, so they cannot serve as a complete parser. They can pre-screen queries or extract simple fragments, but any production use requires a proper parser as the primary mechanism.

4.2.7 ANTLR4

ANTLR4 (ANother Tool for Language Recognition, version 4) is a parser generator that produces top-down, Adaptive LL(*) recursive-descent parsers from context-free grammar specifications written in the ANTLR meta-language [51, 52, 54, 55]. Given a `.g4` grammar file the tool generates a lexer, a parser, and listener/visitor scaffolding in the target language (Python in this project).

The implementation uses Trino's SQL grammar (`TrinoLexer.g4` / `TrinoParser.g4`) from the ANTLR community grammars project. This grammar already contains the `patternRecognition` productions needed for the `MATCH_RECOGNIZE` sub-clauses, as shown in Listing 4.3. The project-specific layer is therefore the Python visitor and validation logic that extracts these parse tree nodes into typed AST objects. A custom error listener intercepts syntax errors and reports them with line/column positions and contextual suggestions.

4.2.8 Implementation of the Parser

4.2.8.1 Designing the Grammar

The implementation uses the Trino SQL grammar obtained from the ANTLR community grammars project (`TrinoLexer.g4` / `TrinoParser.g4`, approximately 1 521 lines across the two grammar files in this project) [56]. This grammar natively contains the `patternRecognition` rule that covers all eight `MATCH_RECOGNIZE` sub-clauses; no modifications to the grammar file were required. Listing 4.3 shows the key productions.

```
patternRecognition
    : aliasedRelation (
        MATCH_RECOGNIZE '('
          (PARTITION BY expression (',' expression)*)?
          (ORDER BY sortItem (',' sortItem)*)?
          (MEASURES measureDefinition (',' measureDefinition)*)?
          rowsPerMatch?
          (AFTER MATCH skipTo)?
          (INITIAL | SEEK)?
          PATTERN '(' rowPattern ')'
          (SUBSET subsetDefinition (',' subsetDefinition)*)?
          (DEFINE variableDefinition (',' variableDefinition)*)?
        ')' (AS? identifier columnAliases?)?
      )?
    ;

measureDefinition : expression AS identifier ;

rowsPerMatch
    : ONE ROW PER MATCH
    | ALL ROWS PER MATCH emptyMatchHandling?
    ;

skipTo
    : SKIP TO NEXT ROW
    | SKIP TO (FIRST | LAST)? identifier
    | SKIP PAST LAST ROW
    ;

rowPattern
    : rowPattern rowPattern             # patternConcatenation
    | rowPattern '|' rowPattern         # patternAlternation
    | patternPrimary patternQuantifier? # quantifiedPrimary
    ;
```

```
35
36 patternPrimary
37     : identifier                                          #
          patternVariable
38     | '(' rowPattern ')'                                  #
          groupedPattern
39     | '{-' rowPattern '-}'                                #
          excludedPattern
40     | PERMUTE '(' rowPattern (',' rowPattern)* ')'        #
          patternPermutation
41     ;
```

Listing 4.3: Key productions of the `patternRecognition` rule in `TrinoParser.g4`

4.2.8.2 Handling Expressions as Strings

`DEFINE` predicates and `MEASURES` expressions contain column references, navigation function calls, arithmetic, comparisons, and selected scalar or aggregate functions. Parsing every expression into a fully typed SQL expression tree would require a much larger expression subsystem. The adopted strategy is therefore to preserve the relevant expression text during parsing and interpret it later through the controlled expression evaluators used by the matching and measure layers. This two-pass strategy keeps the main grammar concise while still avoiding direct execution of raw user expressions (see Section 4.2.12).

4.2.8.3 Parse Tree vs. Abstract Syntax Tree

Parse trees produced by ANTLR4 contain many structural nodes (keywords, punctuation) that are unnecessary for execution. A visitor traverses the parse tree and produces a compact *Abstract Syntax Tree* (AST) whose node types correspond directly to semantic constructs [42, 57]. The key node types are summarised in Table 4.2.

The AST is validated for semantic correctness before pattern compilation: unresolved pattern-variable references, invalid subset definitions, incompatible output modes, and illegal pattern syntax are detected and reported at this stage. Pattern variables without an

Table 4.2: Principal AST node types

Node Type	Represents
`FullQueryAST`	Root of the full query (SELECT + FROM + MR)
`MatchRecognizeClause`	Root of the `MATCH_RECOGNIZE` sub-clause
`PartitionByClause`	`PARTITION BY` column list
`OrderByClause`	`ORDER BY` sort items (`SortItem`)
`MeasuresClause`	`MEASURES` list (`Measure` nodes)
`RowsPerMatchClause`	`ONE / ALL ROWS PER MATCH` mode
`AfterMatchSkipClause`	`AFTER MATCH SKIP` policy
`PatternClause`	`PATTERN` string with tokenizer
`SubsetClause`	`SUBSET` variable definitions
`DefineClause`	`DEFINE` list (`Define` nodes)

explicit `DEFINE` predicate remain valid and are assigned an implicit always-true predicate downstream.

4.2.9 Visitor Implementation

ANTLR4 generates an abstract visitor base class from the grammar [58]. The parsing module is structured as two cooperating visitor classes built on the generated `TrinoParserVisitor` base.

`MatchRecognizeExtractor`. The inner visitor overrides `visitPatternRecognition` and extracts the available sub-clauses in grammar order: `PARTITION BY` column expressions, `ORDER BY` sort items with direction and nulls handling, `MEASURES` definitions (handling `RUNNING`/`FINAL` prefix and alias via a dedicated `smart_split()` helper that correctly traverses nested parentheses and quoted identifiers), rows-per-match mode, after-match-skip policy, the raw pattern string, `SUBSET` definitions, and `DEFINE` variable–condition pairs. After extraction, the parser applies validation routines such as `validate_clauses`, `validate_identifiers`, `validate_pattern_clause`, `validate_pattern_variables_defined`, and `validate_function_usage`. These checks verify required clauses, balanced pattern syntax, incompatible output-mode combinations such as exclusions with `WITH UNMATCHED ROWS`, table-prefix misuse, skip

targets, and malformed uses of supported navigation or aggregate functions. Pattern variables that appear in PATTERN but not in DEFINE are intentionally allowed and later receive the implicit always-true predicate required by the standard.

FullQueryExtractor. The outer visitor handles the top-level singleStatement rule. It extracts the outer SELECT column list, FROM table reference, and any trailing ORDER BY clause via regex, then locates the PatternRecognitionContext node through a recursive tree walk and delegates to MatchRecognizeExtractor.

Public API. The entry point for all callers is parse_full_query(query: str) → FullQueryAST. This function preprocesses the query string (e.g. rewriting empty IN () predicates to a sentinel value to satisfy the grammar), constructs InputStream → TrinoLexer → CommonTokenStream → TrinoParser, replaces the default ANTLR4 error listeners with a CustomErrorListener that raises a structured ParserError carrying exact line, column, and source snippet on any syntax violation, and visits the parse tree with FullQueryExtractor to produce the typed FullQueryAST.

4.2.10 Testing and Evaluation of the Parser

The parser implementation is validated through a structured test suite located in the tests/ directory. Tests are organized into parser-focused checks, integration tests that exercise the parse_full_query pipeline, and SQL:2016-oriented tests derived from reference scenarios and observed Trino behavior.

Parser-focused tests. The parser tests exercise the extraction and validation behavior of the main MATCH_RECOGNIZE sub-clauses, including MEASURES expressions with RUNNING/FINAL keywords, after-match skip targets, output-mode combinations, and pattern syntax. The validation chain is also tested adversarially: supplying an undefined pattern-variable reference, a prohibited function form in MEASURES, an invalid skip target,

or an incompatible output-mode combination triggers a `ParserError` or an empty-result path as appropriate. By contrast, a pattern variable without a corresponding `DEFINE` predicate is valid and receives an implicit always-true condition.

Integration tests. The file `test_match_recognize.py` is adapted from Trino's `TestRowPatternMat` and exercises representative end-to-end scenarios: simple queries, pattern quantifiers (`*`, `+`, `?`, bounded `{n,m}`), pattern exclusion syntax (`{- ... -}`), the supported after-match-skip modes, one-row and all-rows output modes, `CLASSIFIER()` and `MATCH_NUMBER()` special columns, navigation functions, partitioned and ordered input, `SUBSET` aliasing, and `RUNNING`/`FINAL` semantics.

SQL:2016 compliance tests. `test_sql2016_compliance.py` contains targeted tests for standard-oriented behavior: pattern syntax, navigation functions, special aggregate functions, partition boundary handling, subset variable resolution, output-mode semantics, and the supported skip policies. Empty-match handling is covered separately in `test_empty_matches.py` with nine test cases including `SHOW EMPTY MATCHES`, `OMIT EMPTY MATCHES`, `WITH UNMATCHED ROWS`, and combinations with alternation and partitioning.

Edge-case tests. Additional test files cover `IN ()` predicate preprocessing (`test_in_predicate.py`), `IS NULL` handling (`test_is_null_handling.py`), anchored patterns (`test_anchor_patterns.py`), `PERMUTE` patterns (`test_permute_patterns.py`), back-references (`test_back_reference.py`), pattern cache correctness and thread safety (`test_pattern_cache.py`), and aggregate-function behavior (`test_production_aggregates.py`).

4.2.11 Discussion of the Parser

The grammar-driven approach provides several practical advantages: the language specification remains in a single readable `.g4` file; ANTLR4 provides structured parse-tree generation and error-recovery mechanisms, while the custom listener reports syntax errors with line and column context; and the generated visitor scaffolding eliminates manual tree traversal boilerplate.

4.2.11.1 Limits

Two limitations are known. First, advanced SQL constructs outside the supported row-pattern subset, such as recursive common table expressions embedded inside predicate subqueries, are not part of the main execution path; the expression subsystem recognizes the constructs needed for `MATCH_RECOGNIZE` evaluation. Second, because the grammar is based on Trino's dialect, vendor-specific extensions of other systems (Oracle `RUNNING` keyword placement differences, Snowflake aggregate variants) may require minor grammar adjustments if cross-dialect compatibility is needed.

4.2.12 Safe SQL Expression Evaluation

`DEFINE` predicates and `MEASURES` expressions are user-supplied strings that must be interpreted at runtime. Directly calling `eval()` on arbitrary user input would create a code-injection vulnerability. The implementation instead translates the supported SQL expression subset into controlled Python expression forms and evaluates them through AST-aware evaluators with explicit operator and function handling. Listing 4.4 shows the simplified principle.

```
1 import ast
2
3 def safe_evaluate(sql_expr, row_data, context):
4     # 1. Translate SQL syntax to Python equivalents
5     py_expr = translate_sql_to_python(sql_expr)
6
7     # 2. Parse into a Python AST (expression mode only)
```

```
    tree = ast.parse(py_expr, mode='eval')

    # 3. Whitelist-validate: allow comparisons, arithmetic,
    #    boolean ops, literals, subscripts -- forbid imports,
    #    function calls (except navigation functions),
       assignments
    ASTSecurityValidator().visit(tree)

    # 4. Evaluate by walking only the approved AST nodes
    evaluator = ControlledExpressionEvaluator(row_data,
       context)
    return evaluator.visit(tree.body)
```

Listing 4.4: Safe SQL expression evaluation using AST whitelisting

The runtime evaluator follows the same principle but uses the project's `ConditionEvaluator` and measure-evaluation helpers: it walks supported AST nodes explicitly and exposes only the current row context, pattern-variable bindings, supported navigation functions (`PREV`, `NEXT`, `FIRST`, `LAST`), and selected scalar or aggregate operations. It does not expose Python built-ins, imports, file-system access, network access, or interpreter internals.

4.3 SEMANTIC VALIDATION

After the AST is constructed, the engine performs a systematic semantic validation pass before any automaton is built. This pass catches errors that are syntactically valid but logically inconsistent with the supported SQL:2016 row-pattern subset, and applies a set of normalization rules that simplify downstream processing.

4.3.1 Schema and Variable Consistency Checks

The validation and execution setup verify the following properties, raising parser or execution errors when a query cannot be mapped safely to the input DataFrame:

- Columns referenced in `PARTITION BY`, `ORDER BY`, `DEFINE`, and `MEASURES` expressions must be resolvable against the input DataFrame during execution setup or expression evaluation.

- Pattern-variable references in `MEASURES` or `DEFINE` expressions must refer to variables declared in the `PATTERN` clause or made available through `SUBSET`. Variables without explicit `DEFINE` predicates are allowed and are treated as implicit always-true variables.

- If a `SUBSET` clause is present, the subset mapping is normalized and made available to later stages. References to subset aliases and their component variables are then resolved where they are used in measures, predicates, or automaton construction.

- The `AFTER MATCH SKIP TO FIRST/LAST` V options reference a pattern variable V that actually exists in the pattern, catching typos at compile time rather than at runtime.

4.3.2 Navigation-Function Validation

Navigation functions impose inter-variable ordering constraints that must be verified before the automaton is constructed. The `validate_navigation_conditions()` function in `src/matcher/condition_evaluator.py` performs this check by building a *variable timeline*: the order in which distinct pattern variables first appear in the pattern string. For each `DEFINE` predicate the validator then checks:

- Cross-variable `NEXT(`$v.c$`)` references must point to a variable that appears later in the pattern timeline. Self-references and unqualified column navigation are handled by the evaluator according to the available row context. If a navigation condition is determined to be invalid for the pattern, the engine returns an empty result set rather than raising an exception, matching the Trino-oriented behavior used in the tests.

- `PREV(v.c, k)` references are accepted when the referenced variable is available in the pattern context; the evaluator also checks boundary conditions during execution.
- Nested navigation expressions (e.g. `NEXT(PREV(A.price, 1), 1)`) are handled by the enhanced navigation-evaluation path; executor-side helper routines also validate nested forms where they are used.

4.3.3 Output-Mode and Skip-Mode Consistency

Several combinations of output mode, skip mode, and pattern features are invalid in the supported SQL:2016-oriented subset. The validator enforces the following rules:

- Pattern exclusion syntax (`{- ...-}`) is incompatible with `ALL ROWS PER MATCH WITH UNMATCHED ROWS`; this combination is rejected with a descriptive error message.
- `AFTER MATCH SKIP TO FIRST/LAST` V requires that V be a pattern variable, not a subset alias. Subset aliases are resolved at automaton-construction time, not at skip-mode computation time, so this restriction is necessary to ensure a deterministic next-start position.

After all checks pass, the validator records the resolved configurationâ€”normalized output mode, skip mode, implicit-variable defaults, and measure semanticsâ€”ready for the logical planning step.

4.4 LOGICAL PLAN CONSTRUCTION

The logical plan translates the validated AST into a flat, canonical data structure that drives all subsequent phases without further reference to the raw SQL text. In the implementation this materializes as a set of typed Python objects extracted from the `MatchRecognizeClause` AST node and passed as explicit arguments to the execution functions. Table 4.3 lists the principal logical-plan attributes and their types.

Table 4.3: Principal logical-plan attributes extracted from the validated AST

Attribute	Type	Semantics
`partition_by`	`List[str]`	Column names for DataFrame partitioning; empty list means single global partition
`order_by`	`List[str]`	Column names for intra-partition sort order
`pattern_text`	`str`	Normalized pattern string forwarded to the tokenizer
`define`	`Dict[str, str]`	Variable → predicate expression mapping; implicit variables (absent from `DEFINE`) are assigned the always-true predicate `"True"`
`subset_dict`	`Dict[str, List[str]]`	Subset alias → constituent variable list, extracted by `extract_subset_dict()`
`measures`	`Dict[str, str]`	Measure alias → expression string
`measure_semantics`	`Dict[str, str]`	Measure alias → `"RUNNING"` or `"FINAL"`, determined per the rules summarised in Table 5.4
`rows_per_match`	`RowsPerMatch`	Output-row multiplicity enum value
`skip_mode`	`SkipMode`	After-match skip strategy enum value
`skip_var`	`Optional[str]`	Target variable for TO FIRST/TO LAST; `None` for the other two modes

4.4.1 Implicit Variable Defaults

SQL:2016 permits pattern variables to be listed in the `PATTERN` clause without a corresponding entry in the `DEFINE` clause. Such variables match any row unconditionally. During logical plan construction the engine identifies these implicit variables by comparing the set of names extracted from the pattern tokens against the keys of the `define` dictionary. Any variable present in the former but absent from the latter is assigned the always-true predicate (represented as the Python string `"True"`), so that later stages can treat all variables uniformly without special-casing implicit ones.

4.4.2 Measure Semantics Resolution

Measure semantics (RUNNING vs. FINAL) can be specified explicitly using the keywords `RUNNING` and `FINAL` in the `MEASURES` clause, or left implicit for the engine to determine. The resolution algorithm (Figure 4.2) scans all measure expressions in order:

1. If any measure carries an explicit `RUNNING` or `FINAL` keyword, the query is in *mixed-semantics mode* and all other measures without an explicit keyword default to FINAL.

2. In single-semantics mode (no explicit keywords):

    - All measures default to FINAL under `ONE ROW PER MATCH`.
    - Under `ALL ROWS PER MATCH`: `LAST()` defaults to RUNNING; `FIRST()`, `PREV()`, and `NEXT()` default to FINAL; aggregate functions `COUNT()`, `SUM()`, and `ARRAY_AGG()` default to RUNNING; all others default to FINAL.

Once the required attributes are resolved, the logical plan is assembled and passed to the pattern-compilation phase described in Chapter 5.

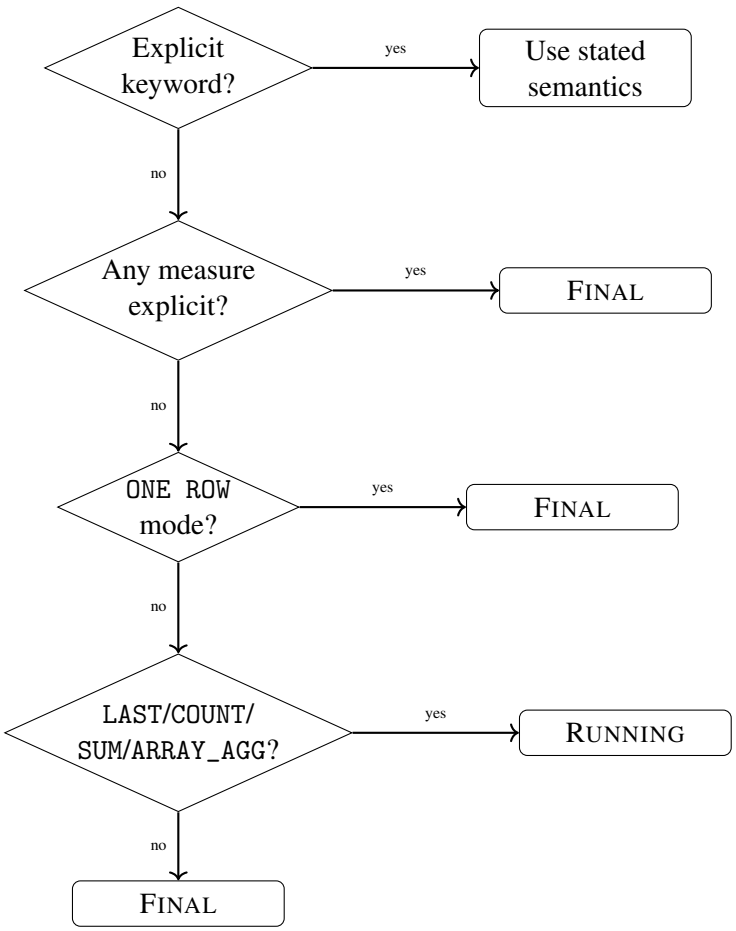

Figure 4.2: Measure semantics resolution algorithm applied to each measure expression individually. The decision tree follows SQL:2016 default-semantics rules.

4.5 CHAPTER SUMMARY

This chapter has presented the system design and methodology of the proposed `MATCH_RECOGNIZE` engine for Pandas DataFrames. It introduced the five-stage architecture—parsing, semantic validation, logical plan construction, pattern compilation, and execution—and explained how the first three stages transform a user query into a validated logical representation.

The chapter first described the overall architecture and then focused on the front-end methodology: SQL parsing, grammar selection, ANTLR4-based parse-tree processing, AST construction, safe expression handling, semantic validation, and logical-plan normalization. These stages establish the boundary between declarative SQL input and the lower-level execution machinery.

Chapter 5 continues from this logical design by describing the automata compilation pipeline, the Pandas execution model, the concrete implementation components, and the optimization techniques used to improve runtime performance and memory behavior.

CHAPTER 5
IMPLEMENTATION AND OPTIMIZATION

This chapter describes how the logical design introduced in Chapter 4 is realized as an executable `MATCH_RECOGNIZE` engine for Pandas DataFrames. The focus is on the implementation layers that translate a validated logical plan into executable pattern matching: automata compilation, pattern tokenization, NFA/DFA construction, Pandas-based execution, measure materialization, and performance optimization.

The chapter is organized as follows. Section 5.1 explains how row patterns are compiled into finite automata. Section 5.2 describes execution over Pandas DataFrames, including partitioning, row classification, skip modes, output modes, and measure evaluation. Section 5.3 summarizes the concrete implementation components. Finally, Section 5.4 presents the optimization layer, including LRU pattern caching, memory management, and enhanced condition evaluation.

5.1 PATTERN COMPILATION WITH FINITE AUTOMATA

Pattern matching in `MATCH_RECOGNIZE` can be modeled as a sequence-recognition problem. A sequence of data rows, each labeled with the pattern variable or variables it satisfies, constitutes a string over the alphabet of pattern variables. The problem of determining whether this string is accepted by the pattern is therefore naturally expressed using finite automata [59].

5.1.1 Non-deterministic Finite Automata (NFA)

A *Non-deterministic Finite Automaton* (NFA) is formally defined as a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where:

- Q is a finite set of *states*;
- Σ is the *input alphabet* (pattern variables in this setting);

- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ is the *transition function*, mapping each state and symbol (or the empty string ε) to a *set* of successor states;
- $q_0 \in Q$ is the *initial state*; and
- $F \subseteq Q$ is the set of *accepting states*.

NFAs naturally model alternation and optional constructs owing to their ability to be in multiple states simultaneously and to use ε-transitions (transitions that consume no input).

Figure 5.1 shows the NFA for pattern $A^+ B?$.

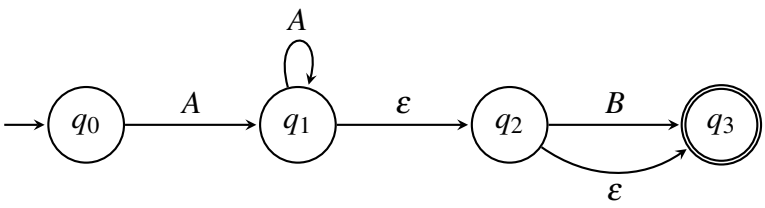

Figure 5.1: NFA for the pattern $A^+ B?$ illustrating ε-transitions and nondeterminism

5.1.2 Thompson's Construction

Thompson's construction algorithm [60] converts a regular expression into an NFA in linear time $O(n)$, where n is the length of the expression. The algorithm proceeds recursively over the expression structure according to the rules in Algorithm 1.

In its classical form, Thompson's construction is linear in the size of the regular expression [42]. The implementation uses the same construction principle, while adding wrapper states, predicate metadata, priorities, and safety guards for SQL-specific constructs such as `PERMUTE`. Consequently, the exact state count depends on the supported SQL pattern constructs rather than on the textbook $2n$ bound alone.

Algorithm 1 Thompson's NFA Construction

procedure BUILD(e)
  if $e = \varepsilon$ **then**
    return NFA with one ε-transition from start to accept
  else if $e = a \in \Sigma$ **then**
    return NFA with one a-labeled transition from start to accept
  else if $e = r \cdot s$ ▷ concatenation **then**
    $N_r \leftarrow$ BUILD(r)
    $N_s \leftarrow$ BUILD(s)
    merge accept state of N_r with start state of N_s
    return combined NFA
  else if $e = r \mid s$ ▷ alternation **then**
    $N_r \leftarrow$ BUILD(r)
    $N_s \leftarrow$ BUILD(s)
    add new start q_{new} with ε-edges to $q_0^{N_r}$ and $q_0^{N_s}$
    add new accept f_{new} with ε-edges from F^{N_r} and F^{N_s}
    return combined NFA
  else if $e = r^*$ ▷ Kleene star **then**
    $N_r \leftarrow$ BUILD(r)
    add ε-edge: start $\to$ accept ▷ zero repetitions
    add ε-edge: $F^{N_r} \to q_0^{N_r}$ ▷ loop back
    return modified NFA
  else if $e = r^{\{n,m\}}$ ▷ bounded quantifier **then**
    construct n mandatory copies followed by $(m-n)$ optional copies
    return combined NFA
  end if
end procedure

5.1.3 Pattern Tokenization

Before an NFA can be constructed the raw pattern string extracted from the SQL query must be converted into a structured, traversable representation. The `tokenize_pattern()` function in `src/matcher/pattern_tokenizer.py` performs this step in a single left-to-right scan, producing a flat list of `PatternToken` objectsâ€”the *linearized* form of the pattern's syntax tree.

PatternToken structure. Each `PatternToken` is a Python `dataclass` with the following key fields:

- `type` â€” a `PatternTokenType` enum value determining which NFA fragment builder is invoked (see Table 5.1);
- `value` â€” the raw textual value extracted from the pattern string (variable name, operator character, etc.);
- `quantifier` â€” optional quantifier string (`'*'`, `'+'`, `'?'`, or `'{n,m}'`) copied from the immediately following quantifier when present; defaults to `None`;
- `greedy` â€” boolean flag indicating greedy (`True`, the default) or reluctant (`False`) quantification, set when a trailing `?` follows the quantifier;
- `metadata` â€” a dictionary carrying type-specific structured data (e.g. a `variables` list for `PERMUTE` tokens, an `excluded_variables` list for `EXCLUSION` tokens); and
- `priority` â€” an integer priority field used by selected builders, especially alternation and `PERMUTE` handling, to order competing transitions; tokens normally default to priority zero unless a construction step assigns a more specific ordering.

Table 5.1: The thirteen `PatternTokenType` values and their roles in the pattern string

Token type	Role in pattern
`LITERAL`	A pattern variable name (`A`, `UP`, `STRT`, etc.)
`ALTERNATION`	The `|` operator
`PERMUTE`	A complete `PERMUTE(...)` construct carrying its variable list in `metadata`
`GROUP_START`	An opening parenthesis `(`
`GROUP_END`	A closing parenthesis `)`
`ANCHOR_START`	The start-of-partition anchor `^`
`ANCHOR_END`	The end-of-partition anchor `$`
`EXCLUSION`	A complete `{- ...-}` exclusion block
`EXCLUSION_START`	Opening delimiter `{-`
`EXCLUSION_END`	Closing delimiter `-}`
`QUANTIFIER`	A standalone quantifier node
`SUBSET`	A `SUBSET` variable reference
`SEQUENCE`	A composite sequence node

Tokenization algorithm. The internal function `_tokenize_pattern_internal()` scans the pattern string character by character, dispatching on the current position:

1. Whitespace is skipped unconditionally.

2. The keyword `PERMUTE` (case-insensitive) triggers `_parse_permute_pattern()`, which collects the comma-separated variable list inside the following parentheses and returns a single `PERMUTE` token whose `metadata['variables']` holds the parsed variable tokens.

3. The two-character sequence `{-` triggers `_parse_exclusion_pattern()`, which reads to the matching `-}` (tracking nesting depth) and returns an `EXCLUSION` token.

4. The structural characters `(`, `)`, `|`, `^`, and `$` each produce a single-character token of the corresponding type.

5. Quantifier characters `*`, `+`, `?`, and `{` are parsed by `parse_quantifier_at()` and *attached* to the preceding token as its `quantifier` and `greedy` fieldsâ€”no separate token is inserted.

6. Identifier characters (`[A-Za-z0-9_]`) are accumulated into a `LITERAL` token.

After the scan, `_optimize_token_sequence()` makes a conservative pass that removes structural redundancies while preserving the one-variable-per-token invariant that the NFA builder requires: adjacent `LITERAL` tokens that look like pattern variable names are deliberately *not* merged.

Quantifier normalization. Quantifiers are stored as attributes of the token they modify rather than as separate tokens. The helper `parse_quantifier()` normalizes all surface forms to the triple $(min, max, greedy)$:

`?`	$\rightarrow$	$(0, 1, \mathit{True})$
`*`	$\rightarrow$	$(0, \infty, \mathit{True})$
`+`	$\rightarrow$	$(1, \infty, \mathit{True})$
`{`n`}`	$\rightarrow$	(n, n, True)
`{`n`,`m`}`	$\rightarrow$	(n, m, True)
append `?` to any	$\rightarrow$	$\mathit{greedy} = \mathit{False}$

This triple is consumed directly by `NFABuilder._apply_quantifier()` during NFA construction.

5.1.4 NFA Construction from Tokens

The `NFABuilder` class in `src/matcher/automata.py` implements the *visitor* pattern over the token list. Its central method `_process_sequence()` iterates the token stream and dispatches on each token's `type` field to invoke the appropriate NFA-fragment builderâ€”mirroring the recursive tree-traversal structure of classical Thompson's construction but applied to the flat linearized token representation rather than an explicit parse tree.

Overall build sequence. The public entry point `NFABuilder.build()` executes the following steps:

1. Check the LRU pattern cache (keyed on an MD5 hash of the token list and DEFINE context) for a previously compiled NFA. A cache hit returns a deep clone of the stored NFA to prevent state sharing.

2. On a cache miss call `_build_nfa_internal()`, which:

   (a) validates anchor semantics (e.g. rejecting patterns such as `A^` where a start anchor follows a literal);

   (b) creates a global start state q_{start} and accept state q_{accept};

   (c) delegates to `_process_sequence()` for the pattern body; and

   (d) connects the pattern fragment to the global states via ε-transitions.

3. Apply `NFA.optimize()` to remove unreachable states, deduplicate transitions, and pre-compute DFA hot-paths.

4. Cache the compiled NFA and return it.

5.1.4.1 Token-to-NFA Fragment Mapping

Table 5.2 summarizes how the visitor translates each token type into NFA structure.

5.1.4.2 Quantifier Expansion

`_apply_quantifier(start, end,` *min*`,` *max*`,` *greedy*`)` wraps a fragment (s, e) in outer envelope states (q_s, q_e) and adds ε-transitions accordingly:

- `?` $(\min = 0, \max = 1)$: adds $\varepsilon(q_s \to s)$ for the match case and $\varepsilon(q_s \to q_e)$ for the skip case.

- $*$ $(\min = 0, \max = \infty)$: adds $\varepsilon(q_s \to s)$, $\varepsilon(e \to q_e)$ (exit), $\varepsilon(q_s \to q_e)$ (skip), and $\varepsilon(e \to s)$ (loop for repetition).

- `+` $(\min = 1, \max = \infty)$: as $*$ but without the skip transition, enforcing at least one match.

Table 5.2: Token types and their NFA fragment builders in `NFABuilder._process_sequence()`

Token type	**NFA construction rule**
`LITERAL`	`create_var_states()` produces a two-state fragment $q_s \xrightarrow{C(v)} q_e$, where $C(v)$ is the callable compiled from the `DEFINE` predicate for variable v; the token's `quantifier` (if any) is applied by `_apply_quantifier()`
`ALTERNATION`	The left branch is finalized; the right branch is processed by a recursive call to `_process_sequence()`; a fresh start state fans out with ε-transitions to both branch heads, and a fresh end state collects both branch tails
`GROUP_START`	Triggers a recursive call to `_process_sequence()` that reads until the matching `GROUP_END`; the group quantifier (if any) is applied to the resulting fragment
`PERMUTE`	`_process_permute()` expands all $n!$ orderings of the n variables and builds a union of $n!$ concatenation fragments under a shared start/end pair
`EXCLUSION`	`_process_exclusion_token()` builds the contained sub-pattern normally but marks every state in the range with `is_excluded = True`; the matcher filters excluded variables from output at match time
`ANCHOR_START/END`	Creates a single anchor state with `is_anchor = True` and the corresponding `anchor_type`; partition-boundary semantics are enforced by the matcher

- `{n,m}`: represents mandatory and optional repetitions using additional states and ε-transitions. The implementation uses a simplified bounded-repetition expansion rather than a complete deep clone of an arbitrary sub-automaton for every repetition.

- **Reluctant** (`greedy = False`): the parser records reluctant quantifiers through the token's `greedy` flag. The NFA construction currently preserves the same structural transitions and logs the reluctant case; the greedy/reluctant distinction is therefore only partially reflected in transition priorities.

5.1.4.3 PERMUTE Expansion

The `PERMUTE`$(v_1, v_2, \ldots, v_n)$ construct is semantically equivalent to the alternation of all $n!$ orderings of its arguments. In the main NFA builder, `_process_permute()` proceeds in three phases:

1. **Complexity guard.** Patterns with more than eight variables are rejected immediately (raising `RuntimeError`) because $8! = 40\,320$ is the largest feasible inline expansion; patterns estimated to produce more than $50\,000$ combinations are redirected to a conservative fallback via `_process_permute_limited()`.

2. **Permutation generation.** The builder generates permutation branches with `itertools.permutations`; optional arguments are expanded through subset-permutation combinations. A separate `ProductionPermuteHandler` utility also exists for reusable cached permutation expansion, but the main automaton construction path performs the branch expansion inside `NFABuilder`.

3. **NFA assembly.** For each permutation $\sigma = (\sigma_1, \ldots, \sigma_n)$, the builder constructs a concatenation fragment by calling `create_var_states()` for each σ_i and connecting the fragments with ε-transitions. All $n!$ concatenation fragments are

then united under a common fork/merge pair with the same ε-alternation structure used for the | operator.

5.1.5 Properties of the Constructed NFA

The NFA returned by `NFABuilder.build()` satisfies the following formal properties:

1. **Controlled size for ordinary patterns.** For ordinary concatenation, grouping, alternation, anchors, and simple quantifiers, the builder adds a small number of states per token or construct. Constructs such as `PERMUTE` can grow factorially, so explicit complexity guards cap the number of expanded branches.

2. **Single accept state.** Exactly one accept state q_{accept} exists; all fragment tails are connected to it via ε-transitions.

3. **Priority-annotated ε-transitions.** Every ε-transition carries an integer priority stored in the `epsilon_priorities` dictionary on its source `NFAState`. Lower numeric values denote higher priority. For greedy quantifiers the loop-back ε has lower priority than the exit ε, so the ε-closure computation (which sorts targets by ascending priority) extends the match before terminating. These priorities are propagated through subset construction so that each DFA transition inherits the minimum (highest-priority) value of the contributing NFA transitions. This priority layer supports the implementation's greedy and leftmost-preference behavior, while some SQL edge cases remain governed by executor-level heuristics.

4. **Thread safety.** Each `NFAState` and the `NFA` object itself are guarded by a `threading.RLock()`. Epsilon-closure results are cached in a bounded dictionary (capped at 1 000 entries) accessed under the same lock, protecting shared automaton data if the execution layer is extended to concurrent partition workers.

5. **Structural validation.** `NFA.validate()` validates transition and ε targets, checks and reports accept-state reachability, and validates general automaton metadata before the NFA is returned to the caller. PERMUTE metadata is also checked for required fields where it appears, but this is a structural consistency check rather than a formal proof of semantic equivalence for every expanded permutation.

5.1.6 Worked Example: Pattern `A B+ C?`

Consider the pattern `A B+ C?` with three pattern variables `A`, `B`, `C` each associated with an arbitrary Boolean DEFINE predicate. The tokenizer produces the flat list:

```
[LITERAL('A'),  LITERAL('B', quantifier='+'),  LITERAL('C',
                        quantifier='?')]
```

The `NFABuilder` then processes this list as follows.

1. A global start state q_0 and accept state q_a are created; q_a is marked `is_accept = True`.

2. **LITERAL `A` (no quantifier).** `create_var_states` allocates q_1 and q_2; adds transition $q_1 \xrightarrow{C(A)} q_2$. The visitor adds $\varepsilon(q_0 \to q_1)$ and advances `current` to q_2.

3. **LITERAL `B` with quantifier `+`.** `create_var_states` allocates q_3, q_4 with $q_3 \xrightarrow{C(B)} q_4$. `_apply_quantifier`$(q_3, q_4, 1, \infty, \mathit{True})$ allocates envelope states q_5, q_6 and adds: $\varepsilon(q_5 \to q_3)$, $\varepsilon(q_4 \to q_6)$ (exit), $\varepsilon(q_4 \to q_3)$ (loop, greedy). The visitor adds $\varepsilon(q_2 \to q_5)$ and advances to q_6.

4. **LITERAL `C` with quantifier `?`.** `create_var_states` allocates q_7, q_8 with $q_7 \xrightarrow{C(C)} q_8$. `_apply_quantifier`$(q_7, q_8, 0, 1, \mathit{True})$ allocates q_9, q_{10} and adds: $\varepsilon(q_9 \to q_7)$ (take C), $\varepsilon(q_8 \to q_{10})$ (exit after C), $\varepsilon(q_9 \to q_{10})$ (skip C, bypass). The visitor adds $\varepsilon(q_6 \to q_9)$ and advances to q_{10}.

5. $\varepsilon(q_{10} \to q_a)$ connects the pattern's tail to the global accept state.

The constructed automaton accepts the language $A \cdot B^{+} \cdot C^{?}$: one row satisfying A, one or more rows satisfying B (the loop $q_4 \to q_3$ implements the + case), and zero or one row satisfying C (the bypass $q_9 \to q_{10}$ implements the ?). In the current implementation, this example produces twelve NFA states after the global start/accept states and quantifier envelopes are included.

After construction the NFA is passed to `DFABuilder.build()` for subset construction as described in the next section.

5.1.7 Deterministic Finite Automata (DFA)

A *Deterministic Finite Automaton* (DFA) is a special case of an NFA in which: (i) there are no ε-transitions, and (ii) for each state q and symbol a, there is at most one successor state in the partial transition relation used by the implementation. DFAs require $O(n)$ time to process an input of length n with no backtracking, making them ideal for row-by-row matching.

Subset Construction (NFA $\to$ DFA). The *subset construction* (powerset construction) algorithm converts an NFA to an equivalent DFA. Each DFA state corresponds to a *set* of NFA states. For a DFA state S and input symbol a:

$$\delta_{\text{DFA}}(S, a) \;=\; \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta_{\text{NFA}}(q, a)\right)$$

where ε-closure(T) is the set of all NFA states reachable from any state in T by zero or more ε-transitions. Although the resulting DFA may have up to $2^{|Q|}$ states in the worst case, patterns arising from `MATCH_RECOGNIZE` syntax are structurally constrained and in practice yield DFAs with far fewer states.

DFA Optimization. After subset construction, the implementation applies iterative optimization passes: unreachable states are removed, equivalent states are merged when their metadata and transitions are compatible, and transitions are grouped for faster

lookup. This reduces memory footprint and matching overhead in practice, but it is not presented as a full canonical Hopcroft minimization.

State Prioritization for SQL Semantics. SQL:2016 mandates *greedy* matching by default: the engine must prefer the longest possible match, and in the case of alternation, the leftmost alternative takes precedence. Standard DFA theory does not encode these preferences directly. For the supported pattern subset, the implementation approximates these preferences by:

1. Assigning a *priority* to each NFA transition based on its position in the pattern and quantifier greediness.

2. Propagating priorities through subset construction so that each DFA transition inherits the minimum (highest-priority) value of the contributing NFA transitions.

3. Ordering outgoing DFA transitions by priority during match execution so that higher-priority paths are considered first in ambiguous cases.

5.2 PANDAS DATAFRAME EXECUTION MODEL

Pandas [13] is a Python library providing the `DataFrame` data structure: a two-dimensional, labeled table whose columns are stored in array-like structures, commonly NumPy or Pandas extension arrays. Its in-memory, column-oriented storage enables vectorized operations over large row sets, making it well suited for the row-by-row DFA simulation that `MATCH_RECOGNIZE` requires.

This section describes how the engine transforms a validated `MATCH_RECOGNIZE` clause and a compiled DFA into concrete output rows. The process passes through six distinct stages: partition, order, row classification, DFA simulation, measure evaluation, and output assembly. Figure 5.2 places these stages in context of the full five-phase architecture introduced at the start of this chapter.

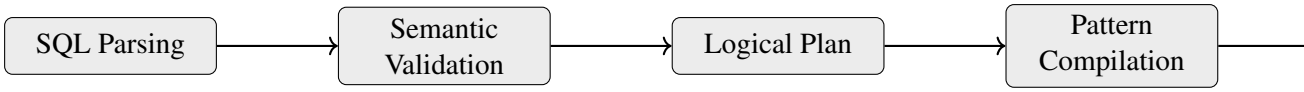

Figure 5.2: Five-phase execution pipeline; the execution stage (shaded) is the subject of this section

5.2.1 Partition and Order

Execution begins by splitting the input DataFrame D into independent *partitions*. The columns named in the `PARTITION BY` clause are passed to `DataFrame.groupby()`, which returns one sub-frame per distinct combination of partition-key values. When no `PARTITION BY` clause is present the entire DataFrame constitutes a single partition.

Each partition is then sorted by the `ORDER BY` columns using `sort_values()`, establishing the temporal order within which the DFA simulation will proceed. The underlying NumPy index is preserved so that row references in navigation functions remain stable.

Because partitions share no state, they can be processed independently. The implementation keeps this partition-level independence explicit, and the optimization layer discussed in Section 5.4.1 provides a parallel-aware scheduling boundary for future execution modes. The remainder of this section describes the processing of a single partition P of n rows, indexed $r_0, r_1, \ldots, r_{n-1}$ in sorted order.

5.2.2 Row Classification

Before the DFA simulation begins, every `DEFINE` predicate is compiled into a callable Python function. The `compile_condition()` function in `src/matcher/condition_evaluator.py` parses the predicate expressionâ€”which may reference pattern variables via dot notation (`A.price`), navigation functions, and SQL comparison operatorsâ€”and wraps it in a closure that accepts a row dictionary and a `RowContext` object. Consistent with Trino's documented behavior, pattern variables that appear in `PATTERN` but are absent

from DEFINE are treated as implicit always-true variables, so they may match any row unless constrained by surrounding pattern structure [6].

The RowContext (Listing 5.1) is the central evaluation context. It provides:

- rows: the full sorted partition as a list of row dictionaries, enabling look-ahead and look-behind via index arithmetic;
- variables: the current match's variable bindings (a mapping from variable name to a list of matching row indices);
- current_idx: the index of the row being tested;
- current_var: the name of the pattern variable whose predicate is being evaluated; and
- subsets: the SUBSET variable definitions.

```python
@dataclass
class RowContext:
    rows:              List[Dict[str, Any]]   # full partition
    variables:         Dict[str, List[int]]   # var -> [row indices]
    current_idx:       int                    # row under test
    current_var:       Optional[str]          # variable being matched
    subsets:           Dict[str, List[str]]   # SUBSET definitions
    defined_variables: Set[str]               # DEFINE variable names
    pattern_variables: List[str]              # all pattern variables
```

Listing 5.1: Key fields of `RowContext` consulted during condition evaluation

Predicates are evaluated lazily during DFA simulation: a condition is called when the matcher attempts the corresponding transition. For larger partitions, the engine enables

a hybrid condition preprocessing path that increases condition-cache capacity and, for very large inputs, pre-warms common condition-evaluation paths on a sample of rows. The codebase also contains a condition-matrix routine for precomputing Boolean arrays, but the main matching path currently relies primarily on the hybrid cache/pre-warming strategy.

5.2.3 DFA Simulation

The core of the execution stage is the DFA simulation loop, shown as Algorithm 2. Starting from the DFA's initial state q_0, the engine processes the sorted partition one row at a time. At each row r_i it evaluates the `DEFINE` predicates for every outgoing transition of the current state, collects the transitions whose predicates evaluate to `True`, and selects the highest-priority one (reflecting greedy or leftmost-match semantics). When the DFA reaches an accepting state, the engine records the current variable bindings as a candidate match and either terminates (for simple patterns) or continues to seek a longer greedy match.

BestTransition selection. When multiple transitions fire simultaneously (e.g. when a row satisfies more than one pattern variable's predicate) the engine applies the following priority ordering, consistent with SQL:2016 greedy-leftmost semantics:

1. Transitions leading to accepting states are preferred over non-accepting ones, ensuring a valid prefix is recorded as a candidate match before later rows are considered.

2. Among non-accepting transitions, those whose source and target state are the same (self-loops for repeating quantifiers such as `A+`) are preferred over state-advancing transitionsâ€”implementing greedy extension.

3. When the pattern contains cross-variable references in `DEFINE` predicates (e.g. `B.price > A.price`), the engine further biases toward the self-loop for the

Algorithm 2 DFA Simulation Loop for a Single Partition

procedure SIMULATEPARTITION(P, *dfa*, *skip_mode*, *config*)
  $results \leftarrow [\,]$
  $i \leftarrow 0$
  while $i < |P|$ **do**
    $q \leftarrow q_0$
    $bindings \leftarrow \{\}$
    $best \leftarrow \bot$
    $j \leftarrow i$
    while $j < |P|$ **do**
      $fired \leftarrow \{(v, q') \mid (q \xrightarrow{v} q') \in dfa, C(v, P[j], ctx_{j,bindings})\}$
      if $fired = \varnothing$ **then**
        break ▷ no transition – end of match attempt
      end if
      $(v^*, q^*) \leftarrow$ BESTTRANSITION($fired$)
      $bindings[v^*] \leftarrow bindings[v^*] \cup \{j\}$
      $q \leftarrow q^*$
      if $dfa.accepting(q)$ **then**
        $best \leftarrow (start{=}i,\ end{=}j,\ bindings$ copy$)$
        if *skip_mode* = TONEXTROW **then**
          break
        end if
      end if
      $j \leftarrow j + 1$
    end while
    if $best \neq \bot$ **then**
      $results$.APPEND($best$)
      $i \leftarrow$ NEXTSTART(i, *best*, *skip_mode*, *bindings*)
    else
      $i \leftarrow i + 1$
    end if
  end while
  return *results*
end procedure

prerequisite variable (`A`) before switching to the dependent one (`B`), ensuring A's rows are fully consumed before B begins.

4. Remaining ties are broken by alternation order: the leftmost alternative in the original `PATTERN` string receives the lowest numeric priority and is therefore selected first.

Variable bindings. Throughout the simulation, `bindings` is a dictionary $\{v \mapsto [i_1, i_2, \ldots]\}$ mapping each pattern variable v to the (sorted) list of partition row indices assigned to it during the current match attempt. At the conclusion of a successful match these lists provide the concrete row sets over which measure expressions and navigation functions are evaluated.

Rows belonging to *exclusion* variables (those wrapped in `{- ...-}`) participate in condition evaluation and state transitions, but the implementation omits them from the `ALL ROWS PER MATCH` output and from the measure-evaluation context.

5.2.4 After Match Skip

After recording a match the engine must decide where to start the next match attempt, controlling whether matches can overlap. SQL:2016 defines four `AFTER MATCH SKIP` strategies; Table 5.3 summarizes their semantics and the corresponding next-start-position computation.

To prevent infinite loops, the engine enforces a monotonic progress invariant: the next scan position is always strictly greater than the current start position. For TO NEXT ROW the engine additionally detects cycles in the sequence of recent start positions and breaks out if the same position is revisited.

5.2.5 Output Modes

SQL row-pattern recognition, including Trino's implementation, exposes two principal output-row multiplicity modes, each admitting further sub-options [6]. The current

Table 5.3: The four `AFTER MATCH SKIP` modes and the resulting next scan position

Mode	Next-start computation
PAST LAST ROW (default)	$i \leftarrow best.end + 1$. No overlap is possible; the scan resumes immediately after the last matched row.
TO NEXT ROW	$i \leftarrow best.start + 1$. Overlapping matches are produced because only a single row is skipped regardless of match length.
TO FIRST V	$i \leftarrow \min(bindings[V])$. The scan resumes at the *first* row assigned to variable V. Requires V to appear in the pattern; enforced at compile time.
TO LAST V	$i \leftarrow \max(bindings[V])$. The scan resumes at the *last* row assigned to variable V. May overlap if V's last row precedes the match end.

engine follows these output multiplicities and applies the effective measure contexts summarized in Table 5.4. Trino documents RUNNING as the default row-pattern expression semantics; this implementation additionally applies function-specific defaults for undecorated measures in `ALL ROWS PER MATCH` mode to match the behavior exercised by the regression suite.

Table 5.4: Output modes and their row-count and measure-semantics behavior

Mode	Rows out	Measure evaluation context
ONE ROW PER MATCH	1 per match	FINAL: measures see the complete, fi ized variable bindings. Navigation fu tions (`FIRST`, `LAST`) operate over the matched span.
ALL ROWS PER MATCH	1 per matched row	Implementation default per function: `L` uses RUNNING; `FIRST`/`PREV`/`NEXT` use NAL; `COUNT`, `SUM`, and `ARRAY_AGG` RUNNING; other aggregate functions fault to FINAL. Explicit `RUNNING`/`FI` keywords override the default.
ALL ROWS . . . WITH UNMATCHED	1 per every input row	Matched rows carry measure values; matched rows emit `NULL` for all meas columns.

One Row Per Match. For each recorded match the engine constructs a single output dictionary. The `PARTITION BY` and `ORDER BY` column values are copied from the first matched row; measure expressions are evaluated once with FINAL semantics (all variable bindings are visible). The resulting dictionary is appended to the results list and later assembled into the output DataFrame.

All Rows Per Match. For each recorded match the engine iterates over the non-excluded rows belonging to the match span (from `best.start` to `best.end` inclusive). For each output row r_j in this filtered span the engine:

1. assigns r_j a `CLASSIFIER` column value equal to the name of the pattern variable to which row j was assigned;
2. evaluates each measure expression with the semantics determined by Table 5.4: RUNNING semantics limits variable bindings to rows $\leq j$, while FINAL semantics always uses the complete bindings; and
3. appends the resulting row dictionary to the results list.

Rows assigned to an exclusion variable are omitted from the `ALL ROWS PER MATCH` output in the current implementation.

5.2.6 Measure Evaluation

Measure expressions are evaluated by the `MeasureEvaluator` class in `src/matcher/measure_eva`. Each declared measure is compiled into a callable similar to the condition callables described in Section 5.2.2. The evaluator dispatches on the expression type:

Navigation functions. `FIRST(`$v.c$`)`, `LAST(`$v.c$`)`, `PREV(`c`, `k`)`, and `NEXT(`c`, `k`)` resolve to direct index lookups into the partition row list:

- `FIRST`$(v.c) \rightarrow P[bindings[v][0]][c]$
- `LAST`$(v.c) \rightarrow P[bindings[v][-1]][c]$

- `PREV`$(c, k) \rightarrow P[j-k][c]$ (where j is the current output row index)
- `NEXT`$(c, k) \rightarrow P[j+k][c]$

All accesses are bounds-checked: out-of-partition references return `NULL` per SQL:2016.

Aggregate functions. `COUNT`, `SUM`, and `AVG` are implemented as running or final aggregates over the row index sets stored in `bindings`. Under RUNNING semantics the aggregate is computed over $\{i \in bindings[v] \mid i \leq j\}$; under FINAL semantics it uses all rows in $bindings[v]$.

Pattern-variable column references. An expression of the form *v.c* (without a function wrapper) resolves to the column c of the *first* row assigned to v. In RUNNING mode the reference is restricted to assigned rows up to the current output position.

Arithmetic and logical expressions. Compound expressions (e.g. `LAST(A.price) - FIRST(A.price)`) are evaluated by recursively evaluating sub-expressions and applying the indicated operator.

RUNNING vs. FINAL semantics. The distinction between the two semantics is central to `ALL ROWS PER MATCH` output. Under RUNNING semantics a measure evaluated for output row j sees only rows $r_0, \ldots, r_j$â€"the match is *in progress* and future rows are unknown. Under FINAL semantics the full match is always available regardless of which row is being output.

Consider a `SUM(A.price)` measure over a match covering rows 3, 5, and 7. In RUNNING mode the output for each row is a cumulative partial sum:

Output row	RUNNING `SUM`
3	$P[3].price$
5	$P[3].price + P[5].price$
7	$P[3].price + P[5].price + P[7].price$

whereas FINAL mode emits the same total on all three output rows.

5.2.7 Worked End-to-End Example

To illustrate the full execution pipeline consider the following query applied to a stock-price DataFrame with columns `symbol`, `date`, and `price`:

```
SELECT *
FROM prices
MATCH_RECOGNIZE (
  PARTITION BY symbol
  ORDER BY date
  MEASURES
    FIRST(A.price) AS start_price,
    LAST(B.price)  AS bottom_price,
    LAST(C.price)  AS end_price
  ONE ROW PER MATCH
  AFTER MATCH SKIP PAST LAST ROW
  PATTERN (A+ B+ C+)
  DEFINE
    A AS PREV(A.price, 1) IS NULL OR A.price >= PREV(A.price,
        1),
    B AS B.price <  PREV(B.price, 1),
    C AS C.price >  PREV(C.price, 1)
)
```

Listing 5.2: Example query: detecting a V-shape price pattern

Suppose the partition for `symbol = 'AAPL'` contains five rows sorted by `date` with prices $100, 102, 98, 95, 103$. Execution proceeds as follows.

Step 1 â€” Partition and Order. `groupby('symbol')` produces a single-symbol sub-frame; `sort_values('date')` confirms the price sequence $[100, 102, 98, 95, 103]$.

Step 2 â€” DFA construction. The pattern `A+ B+ C+` compiles to an NFA with 16 states in the current implementation and then to a DFA with 4 states via subset construction (Section 5.1.7).

Step 3 â€” Simulation from $i = 0$.

- Row 0 ($p = 100$): `PREV(A.price)` is undefined at the first row, so the explicit `IS NULL` guard in A's predicate is satisfied. Transition $q_0 \xrightarrow{A} q_1$ fires; $bindings[A] = [0]$. State q_1 is not accepting.

- Row 1 ($p = 102 \geq 100$): `A` predicate satisfied; self-loop $q_1 \xrightarrow{A} q_1$. $bindings[A] = [0,1]$.

- Row 2 ($p = 98 < 102$): `A` fails, `B` passes ($p < prev$). Transition $q_1 \xrightarrow{B} q_2$. $bindings[B] = [2]$.

- Row 3 ($p = 95 < 98$): `B` satisfied; self-loop $q_2 \xrightarrow{B} q_2$. $bindings[B] = [2,3]$.

- Row 4 ($p = 103 > 95$): `C` satisfied; transition $q_2 \xrightarrow{C} q_3$. q_3 is accepting. $best =$ ($start$=0, end=4, $\{A$:$[0,1]$, B:$[2,3]$, C:$[4]\}$).

Step 4 â€" After Match Skip. Mode is PAST LAST ROW: next scan starts at $i = 5$, which equals $|P|$, so the simulation terminates.

Step 5 â€" Measure evaluation (ONE ROW PER MATCH, FINAL).

$$\texttt{start\_price} = \texttt{FIRST}(A.price) = P[0][price] = 100$$

$$\texttt{bottom\_price} = \texttt{LAST}(B.price) = P[3][price] = 95$$

$$\texttt{end\_price} = \texttt{LAST}(C.price) = P[4][price] = 103$$

Step 6 â€" Output assembly. A single output row is emitted: `(symbol='AAPL', start_price=100, bottom_price=95, end_price=103)`.

This example demonstrates how all execution components–partition ordering, DFA simulation with greedy quantifiers, variable binding accumulation, after-match skip, and measure evaluationâ€"interact to produce a single output row for the supported SQL:2016 row-pattern subset from a five-row price sequence.

5.3 IMPLEMENTATION COMPONENTS

The implementation provides a `MATCH_RECOGNIZE` engine for Pandas DataFrames following the supported SQL:2016 row-pattern semantics for sequential data. The implementation has four core components: an ANTLR4-based SQL parser that transforms `MATCH_RECOGNIZE` queries into abstract syntax trees; a finite-automata engine that converts pattern expressions into optimized NFAs and DFAs using Thompson's construction; a pattern-matching executor that processes DataFrames with support for partitioning and ordering; and a measure evaluator that handles navigation functions (`PREV`, `NEXT`, `FIRST`, `LAST`) and aggregations with both running and final semantics. The system supports advanced pattern features including anchors, quantifiers, alternation, permuted patterns, and exclusion syntax.

5.3.1 ANTLR4 Grammar and AST Construction

The system uses the Trino SQL grammar from the ANTLR community grammars project and extracts its `MATCH_RECOGNIZE` productions into project-specific AST nodes. This ANTLR4 grammar covers the pattern constructs used by the implementation, including nested patterns, alternation, quantifiers, grouping, and exclusions [56]. The parser's output is a detailed parse tree that captures the syntactic structure of each SQL query [40, 39].

5.3.2 Finite Automata

Pattern Tokenization. Pattern expressions are tokenized using a single-pass tokenization routine that supports anchors, grouping, quantifiers, and nested constructs.

NFA Construction. Thompson's construction algorithm is used to translate pattern tokens into a Non-deterministic Finite Automaton (NFA). Each state captures transitions, variable bindings, anchors, and metadata for advanced features. Quantifiers (`*`, `+`, `?`, $\{n,m\}$) are directly mapped to sub-automata [29].

DFA Construction and Optimization. The NFA is systematically converted into a Deterministic Finite Automaton (DFA) using subset construction, reducing the number of active states that must be tracked during execution. Optimization steps include removal of unreachable states, compatible-state merging, and transition grouping. The resulting DFA supports efficient row-by-row evaluation while preserving pattern semantics, priorities, and custom conditions for the supported pattern semantics [61, 62, 29].

5.3.3 Execution

The matching engine applies the DFA sequentially over each partition of the DataFrame with specialized handling for anchors, skip modes, and edge cases. The executor coordinates the entire pattern-matching process, integrating parsing, automata construction, match identification, and result formatting.

Condition and Measure Evaluation. A dedicated evaluation subsystem translates SQL pattern conditions and measures into controlled Python callables and evaluator routines that support navigation functions and aggregations with SQL-style `NULL` handling for both running and final calculations.

5.4 PERFORMANCE OPTIMIZATION TECHNIQUES

5.4.1 Parallel Partition Processing

Because `MATCH_RECOGNIZE` evaluates each `PARTITION BY` group independently, partitions provide a natural unit of parallel work. The implementation includes a `ParallelExecutionManager` in `src/utils/performance_optimizer.py`, which estimates whether partition-level work is large enough to justify parallel execution. The current top-level `match_recognize` path uses this manager as a parallel-aware scheduling layer but still processes the partition work sequentially after ordering each partition. The design therefore preserves the partition independence needed for future parallel dispatch without changing match semantics in the current execution path.

5.4.2 LRU Pattern Caching

Pattern compilation—tokenization → NFA construction → DFA conversion → optimization—is computationally expensive, especially for patterns with quantifiers and `PERMUTE` constructs. The implementation maintains a thread-safe *Least Recently Used* (LRU) cache that maps a hash of the pattern string and `DEFINE` predicates to the compiled pattern (the full NFA and DFA pipeline result). Key configuration parameters are listed in Table 5.5.

Table 5.5: Pattern cache configuration defaults (`src/config/production_config.py`)

Parameter	Default	Description
`cache_size_limit`	50 000 entries	Maximum cached patterns
`cache_memory_limit_mb`	2 048 MB	Maximum cache memory
`cache_ttl_seconds`	3 600 s	Entry time-to-live
`cache_clear_threshold_mb`	1 600 MB	Proactive eviction threshold

5.4.3 Memory Management

Large-scale matching over millions of rows requires careful memory management. The implementation treats partitions as independent execution units and iterates over them separately during matching. In the current executor, however, partition objects are materialized from the Pandas `groupby()` result before they are processed, so the memory model is partition-oriented rather than fully streaming. Memory usage is monitored via `psutil`; garbage collection is triggered proactively when consumption approaches user-configured limits (`cache_clear_threshold_mb`). An object-pool layer (`src/utils/memory_management.py`) reuses frequently allocated objects across successive match operations, reducing allocator pressure on long-running queries.

5.4.4 Enhanced Condition Evaluation

For larger partitions, the engine reduces repeated Python function-call overhead through smart condition preprocessing. The main path in `EnhancedMatcher._smart_condition_pre` enlarges the condition-evaluation cache when the partition contains more than 5 000 rows and pre-warms common condition paths on a sample when the partition contains more than 50 000 rows. The codebase also contains `EnhancedMatcher._vectorize_condition_evaluati` which can materialize a Boolean condition matrix, but this is not the primary path used by the current matcher.

Table 5.6 summarizes the performance optimizations and the conditions under which each is applied.

Table 5.6: Performance optimizations and their activation conditions

Technique	**Activation condition**	**Primary benefit**
LRU pattern cache	Always on	Eliminates redundant NFA/DFA construction across repeated queries
Parallel-aware partition scheduling	Multiple partitions, sufficient rows, and acceptable memory pressure	Preserves a scheduling boundary for future concurrent partition execution
Enhanced condition cache	Partition size $>$ 5 000 rows	Increases cache capacity for repeated predicate evaluation
Condition pre-warming	Partition size $>$ 50 000 rows	Warms common predicate paths on a sample of rows before the full scan
Partition-oriented processing	Always on	Keeps match state partition-local, although Pandas group objects may still be materialized before processing
Proactive GC / memory guard	Process RSS $\geq$ `cache_clear_threshold_mb`	Prevents OOM errors on very large dataset scans

5.5 CHAPTER SUMMARY

This chapter has presented the implementation and optimization of the `MATCH_RECOGNIZE` engine. It explained how row patterns are tokenized, compiled into NFAs, converted into DFAs, and executed over ordered Pandas partitions. It also described the implementation of after-match skip behavior, output modes, measure evaluation, and the runtime components that connect the parser and logical plan to the matching engine.

The chapter also described the optimization layer: LRU-style pattern caching to avoid repeated compilation, parallel-aware partition scheduling, partition-oriented memory management, and enhanced condition caching/pre-warming to reduce repeated predicate overhead. These optimizations are orthogonal to the core matching semantics and can be evaluated independently in the experimental chapter.

Chapter 6 evaluates these design and implementation decisions empirically, measuring correctness, throughput, memory behavior, and performance across the selected pattern families and dataset sizes.

CHAPTER 6
EXPERIMENTAL EVALUATION

This chapter evaluates the proposed `MATCH_RECOGNIZE` implementation across five dimensions: output correctness on selected Trino-aligned reference queries, coverage of the supported SQL:2016 row-pattern subset, row-pattern type coverage, performance and scalability, and developer usability. Section 6.1 describes the evaluation methodology, including the hardware environment and the metrics used. Section 6.2 presents the benchmark dataset and test patterns. Section 6.3 reports correctness results. Section 6.4 presents performance results. Section 6.5 compares the system with existing tools.

6.1 EVALUATION METHODOLOGY

This chapter evaluates the proposed `MATCH_RECOGNIZE` implementation through five complementary criteria.

1. **Correctness.** Results are compared row-by-row with Trino across selected pattern types to check matched rows, pattern alignment, and measure calculations for the supported feature subset.

2. **SQL:2016 feature coverage.** Support for key `MATCH_RECOGNIZE` clauses is verified against the implemented subset, including `PARTITION BY`, `ORDER BY`, `PATTERN`, `DEFINE`, `MEASURES`, `SUBSET`, output modes (`ONE ROW PER MATCH` / `ALL ROWS PER MATCH`), after-match skip policies, and navigation functions [2, 3, 38].

3. **Pattern-type coverage.** A variety of row-pattern types are exercised, including simple sequences, quantified repetitions (`*`, `+`, `?`, `{n,m}`), ascending/descending trends, recovery patterns, alternation, exclusion blocks (`{- ... -}`), anchored patterns, and `PERMUTE` constructs.

4. **Performance and scalability.** Execution time is measured across increasing DataFrame sizes and pattern complexities to characterize how the engine scales with data volume and pattern structure.

5. **Usability (developer effort).** The lines of code required to express equivalent logic as idiomatic Pandas serve as a baseline. Hand-written Pandas solutions typically require substantially more code (e.g. chained `groupby`/`shift`/joins) than a single `MATCH_RECOGNIZE` statement, so reducing LOC is a core goal of the RPR approach [2, 3, 23, 63].

6.1.1 Environment and Configuration

The experiments were performed on a workstation running Ubuntu 24.04.2 LTS with an Intel Core i7-11800H (8 physical cores, 16 hardware threads, base frequency 2.30 GHz) and 16 GB of DDR4 RAM at 3200 MHz.

The software environment comprised:

- **Baseline SQL engine:** Trino 473.
- **Runtimes:** Java 11.0.25 LTS and Python 3.12.7.
- **Core Python libraries:** `pandas` 2.2.3, `numpy` 1.26.4, `antlr4-python3-runtime` 4.13.2, `antlr4-tools` 0.2.1, and `pytest` 8.3.4.
- **Analysis and visualization tooling:** Jupyter, `unittest`, `matplotlib` 3.10.0, `plotly` 5.24.1, and `seaborn` 0.13.2.
- **Standard library modules used:** `typing`, `dataclasses`, `re`, `operator`, `math`, `itertools`, `collections`, `enum`, `time`, `sys`, `os`, `logging`, and `traceback`.

The `MATCH_RECOGNIZE` implementation uses ANTLR4 for parsing Trino SQL, `pandas` for DataFrame management, and custom AST/NFA/DFA modules for pattern detection (see Chapter 4).

6.1.2 Evaluation Metrics

Each test case is assessed on the five dimensions defined above.

Correctness. The result DataFrame produced by the implementation is compared with the result set returned by Trino using an exact row-level equality check—same rows, same column values, and same order where order is deterministic.

SQL:2016 feature coverage. Each clause and sub-feature of `MATCH_RECOGNIZE` is marked as *supported*, *partially supported*, or *unsupported* based on whether the implementation supports the feature and whether the exercised behavior matches the Trino-aligned reference tests.

Pattern-type coverage. Test cases are grouped by row-pattern type (sequence, repetition, trend, recovery, alternation, exclusion, anchor, `PERMUTE`). A pattern type is considered covered if at least one test case exercises it and produces correct output.

Performance and scalability. Execution time is recorded as wall-clock seconds for each pattern–size run. Summary tables report averages across pattern families or dataset sizes, and the resulting measurements are used to characterize how the engine scales with data volume and pattern structure.

Usability. The number of non-blank, non-comment lines in the equivalent idiomatic Pandas solution is recorded and compared with the single-statement `MATCH_RECOGNIZE` query, providing a concrete measure of developer effort reduction.

6.2 DATASET AND TEST PATTERNS

6.2.1 Dataset

An `orders` table is used to demonstrate and validate pattern-matching queries. The dataset comprises three attributes [7]: `customer_id` (`VARCHAR`), which identifies the

customer (two distinct values: `cust_1` and `cust_2`); `order_date` (`DATE`), the date the order was placed (ranging from 2020-05-11 to 2020-05-18); and `price` (`INTEGER`), the purchase amount on that date. Table 6.1 shows the eight records used in the correctness demonstrations.

Table 6.1: Sample `orders` dataset (`customer_id`, `order_date`, `price`)

customer_id	order_date	price
cust_1	2020-05-11	100
cust_1	2020-05-12	200
cust_2	2020-05-13	8
cust_1	2020-05-14	100
cust_2	2020-05-15	4
cust_1	2020-05-16	50
cust_1	2020-05-17	100
cust_2	2020-05-18	6

Each customer appears multiple times in chronological order: `cust_1` spans five dates and `cust_2` spans three. This structure enables focused validation of partitioning, ordering, and pattern detection (e.g. sequences of rising or falling prices per customer). In the experiments, data are loaded via a `VALUES` clause in Trino and checked against the Pandas implementation.

```
CREATE TABLE memory.default.orders (
    customer_id VARCHAR,
    order_date  DATE,
    price       INTEGER
);
```

Listing 6.1: Creating the in-memory table in Trino

```
INSERT INTO memory.default.orders VALUES
    ('cust_1', DATE '2020-05-11', 100),
    ('cust_1', DATE '2020-05-12', 200),
    ('cust_2', DATE '2020-05-13',   8),
    ('cust_1', DATE '2020-05-14', 100),
    ('cust_2', DATE '2020-05-15',   4),
    ('cust_1', DATE '2020-05-16',  50),
```

```
8     ('cust_1', DATE '2020-05-17', 100),
9     ('cust_2', DATE '2020-05-18',   6);
```

Listing 6.2: Inserting data into the table

6.3 CORRECTNESS RESULTS

This section assesses output correctness by comparing results from three independent approaches—the proposed engine, Trino 473, and a hand-written Pandas baseline—on the same SQL `MATCH_RECOGNIZE` query and dataset.

6.3.1 Experimental Setup

The correctness experiments use the same hardware and software configuration described in Section 6.1.1. For each reference query, the input data are loaded into both Trino and a Pandas DataFrame with equivalent schemas. The result sets are then compared by row order and measure values after normalizing case-only column-name differences between SQL and Pandas outputs.

6.3.2 Use Case: Stock Price Analysis

The `stock_price` table is used to demonstrate and validate pattern-matching queries with the `MATCH_RECOGNIZE` clause. The table includes three columns: `company`, `price_date`, and `price`, representing a history of daily stock prices for market analysis. Here, `company` specifies the company name, `price_date` is the trading date, and `price` records the stock's value on that day [15].

This example detects bullish momentum by identifying a three-row window in which the second and third rows increase relative to their immediate predecessors. The query partitions records by company (`PARTITION BY company`), orders them chronologically (`ORDER BY price_date`), and applies `PATTERN (A B C)` with `DEFINE` predicates on `B` and `C`. The variable `A` has no explicit `DEFINE` predicate and therefore uses the standard implicit always-true behavior. Key attributes—starting price, ending price, and match

ID—are produced in the MEASURES clause. A structured pandas DataFrame serves as input to MATCH_RECOGNIZE, enabling automated detection of upward trends across firms for the supported row-pattern subset. Additional use cases are available in the project repository [38].

Table 6.2: The tabular form of the stock_price table

Company	Price_date	Price
A	2020-10-01	50
B	2020-10-01	89
A	2020-10-02	36
B	2020-10-02	24
A	2020-10-03	39
B	2020-10-03	37
A	2020-10-04	42
B	2020-10-04	63
A	2020-10-05	30
B	2020-10-05	65
A	2020-10-06	47
B	2020-10-06	56
A	2020-10-07	71
B	2020-10-07	50
A	2020-10-08	80
B	2020-10-08	54
A	2020-10-09	75
B	2020-10-09	30
A	2020-10-10	63
B	2020-10-10	32

Load the dataset as a Pandas DataFrame.

```
import pandas as pd
data = {
    "Company": [
        "A","B","A","B","A","B","A","B","A","B",
        "A","B","A","B","A","B","A","B","A","B"
    ],
    "Price_date": [
        "2020-10-01","2020-10-01","2020-10-02","2020-10-02","
           2020-10-03",
```

```
        "2020-10-03","2020-10-04","2020-10-04","2020-10-05","
           2020-10-05",
        "2020-10-06","2020-10-06","2020-10-07","2020-10-07","
           2020-10-08",
        "2020-10-08","2020-10-09","2020-10-09","2020-10-10","
           2020-10-10"
    ],
    "Price": [
        50,89,36,24,39,37,42,63,30,65,
        47,56,71,50,80,54,75,30,63,32
    ]
}
df = pd.DataFrame(data)
df["Price_date"] = pd.to_datetime(df["Price_date"])
```

Listing 6.3: Python baseline dataset.

Using the proposed `MATCH_RECOGNIZE`.

```
from src.executor.match_recognize import match_recognize

query = """
SELECT *
FROM stock_price
MATCH_RECOGNIZE (
  PARTITION BY Company
  ORDER BY Price_date
  MEASURES
    FIRST(A.Price) AS start_price,
    LAST(A.Price)  AS end_price,
    MATCH_NUMBER() AS match_id
  PATTERN (A B C)
  DEFINE
    B AS B.Price > PREV(B.Price),
    C AS C.Price > PREV(C.Price)
); """
result = match_recognize(query, df)
print(result)
```

Listing 6.4: Using the proposed `MATCH_RECOGNIZE`.

Results (proposed engine):

```
  Company  start_price  end_price  match_id
0       A           36         36         1
1       A           30         30         2
2       B           24         24         1
```

Using Trino and Oracle as Baselines.

In the second step, the same pattern-matching query was applied using the same data in the Trino environment to validate the correctness of the proposed approach. The stock price history was created as an in-memory table using `CREATE TABLE` and populated through `INSERT INTO`, ensuring that the schema and data (company, date, and price) remained consistent with the Pandas DataFrame used earlier. The `MATCH_RECOGNIZE` query was then executed in Trino to detect the same three-row increasing-window pattern per company. In addition, the same query was validated in Oracle Database via `MATCH_RECOGNIZE` [15, 64], using an equivalent `stock_price` table with the same schema and rows; the Oracle results matched the expected behavior, strengthening cross-system validation under standard `MATCH_RECOGNIZE` semantics [64].

```
CREATE TABLE memory.default.stock_price (
    Company    VARCHAR,
    Price_date DATE,
    Price      INTEGER
);
```

Listing 6.5: Creating the in-memory table in Trino.

```
INSERT INTO memory.default.stock_price (Company, Price_date,
   Price) VALUES
('A', DATE '2020-10-01', 50),
('B', DATE '2020-10-01', 89),
('A', DATE '2020-10-02', 36),
('B', DATE '2020-10-02', 24),
...
('A', DATE '2020-10-10', 63),
('B', DATE '2020-10-10', 32);
```

Listing 6.6: Inserting data into the table.

```
SELECT *
FROM memory.default.stock_price
MATCH_RECOGNIZE (
  PARTITION BY company
  ORDER BY price_date
  MEASURES
    FIRST(A.price) AS start_price,
    LAST(A.price)  AS end_price,
    MATCH_NUMBER() AS match_id
  PATTERN (A B C)
  DEFINE
    B AS B.price > PREV(B.price),
    C AS C.price > PREV(C.price)
);
```

Listing 6.7: Executing the same `MATCH_RECOGNIZE` query in Trino.

Results (Trino):

```
 company | start_price | end_price | match_id
---------+-------------+-----------+----------
 A       |          36 |        36 |        1
 A       |          30 |        30 |        2
 B       |          24 |        24 |        1
```

Python DataFrame Baseline. As a Python baseline, the same pattern logic was implemented directly in `pandas`, without using `MATCH_RECOGNIZE`. The input DataFrame (Listing 6.3) is converted to `datetime` and sorted by company and trading date. For each company, the baseline computes a Boolean signal `inc := diff(Price) > 0` and marks candidate starts where the next two differences are positive, corresponding to the SQL pattern rows A, B, C with $B > A$ and $C > B$. Candidates are processed sequentially with a non-overlapping policy, advancing `next_start_pos = i + 3` to emulate `AFTER MATCH SKIP PAST LAST ROW`. For each accepted start, the implementation outputs `company`, `match_id`, and measures `start_price` and `end_price` from the start row (A), returning the resulting matches as a `DataFrame` [63].

```
import pandas as pd
```

```python

def detect_two_consecutive_increases(
    df: pd.DataFrame,
    company_col: str = "Company",
    date_col:    str = "Price_date",
    price_col:   str = "Price",
) -> pd.DataFrame:
    gdf = df.copy()
    gdf[date_col] = pd.to_datetime(gdf[date_col])
    gdf = gdf.sort_values([company_col, date_col]).
       reset_index(drop=True)
    out_rows = []
    for company, g in gdf.groupby(company_col, sort=False):
        g = g.reset_index(drop=True)
        inc = g[price_col].diff() > 0
        start_mask = (inc.shift(-1).fillna(False)
                      & inc.shift(-2).fillna(False))
        candidates = [int(i) for i, ok in enumerate(
           start_mask)
                      if ok and i + 2 < len(g)]
        match_id, next_start_pos = 0, 0
        for i in candidates:
            if i < next_start_pos:
                continue
            match_id += 1
            out_rows.append({
                "company":     company,
                "start_price": g.loc[i, price_col],
                "end_price":   g.loc[i, price_col],
                "match_id":    match_id,
            })
            next_start_pos = i + 3    # AFTER MATCH SKIP PAST
               LAST ROW
    return pd.DataFrame(out_rows)

result = detect_two_consecutive_increases(df)
print(result)
```

Listing 6.8: Equivalent pattern matching in Python/pandas.

Results (Python/pandas):

```
  company  start_price  end_price  match_id
0       A           36         36         1
1       A           30         30         2
2       B           24         24         1
```

For this reference scenario, the proposed engine, Trino 473, and the hand-written Pandas baseline produce identical output. This provides evidence that partitioning, ordering, implicit `DEFINE` behavior, navigation with `PREV`, `MATCH_NUMBER()`, and measure materialization are handled correctly for this supported pattern family.

6.4 PERFORMANCE RESULTS

Performance was evaluated on the Amazon UK Products Dataset using five pattern families and eight input sizes (25 K, 35 K, 50 K, 75 K, 100 K, 150 K, 200 K, and 300 K rows). The complexity model, execution-time, throughput, and memory results are reported in Sections 6.6.1–6.6.6. Across the 40 measured pattern–size runs, the engine completed all tests successfully, with an average throughput of approximately 9 444 rows/s and a total measured execution time of about 550.34 s. The results show predictable scaling over the evaluated range: simpler patterns remain faster, while quantified and nested patterns incur higher transition and condition-evaluation overhead.

6.5 COMPARISON WITH EXISTING SYSTEMS

The evaluated system is not intended to replace full SQL engines such as Oracle or Trino, nor streaming platforms such as Flink. Its design goal is narrower: it brings a practical subset of SQL row-pattern matching into in-memory Python analytics workflows over `pandas` DataFrames.

Compared with Trino and Oracle, the proposed implementation avoids the need to load data into a database server before running `MATCH_RECOGNIZE`. This makes it convenient for notebook and DataFrame-centered analysis, but it also means that the implementation supports only the subset described in Chapters 4 and 5. Trino and Oracle

remain more complete SQL systems with mature optimizers, distributed execution, and broader standard coverage. Compared with hand-written Pandas logic, the SQL form is more declarative and reusable: partitioning, ordering, pattern variables, skip behavior, and measures are expressed in one query rather than in custom procedural code for each pattern.

6.6 USE CASES AND APPLICATIONS

The `MATCH_RECOGNIZE` implementation is applicable to several sequential-analysis settings, such as log analysis, time-series processing, and financial sequence analysis, where recurring row patterns are important.

To assess correctness, each pattern-matching query was executed using three independent approaches. First, a **Python-UDF baseline**: custom functions were implemented—built from `groupby`, `rolling`, and `shift`—to detect the same sequences directly in `pandas` DataFrames; this required adjusting the code for each query to reproduce the intended semantics [63]. Second, a **Trino baseline**: the equivalent `MATCH_RECOGNIZE` statement was run in Trino (version 473) and treated its output as the reference result. Third, the **proposed engine**: `MATCH_RECOGNIZE` was executed on the same SQL string and DataFrame; the system parses the query, compiles a DFA, and performs the matching within `pandas`.

Throughout the evaluated scenarios, the results from the three approaches were compared on a row-by-row basis. For the reference cases reported in this chapter, the `MATCH_RECOGNIZE` output matched the Trino result (see Table 6.3). In contrast, the UDF method involved significantly more code and was more prone to errors during development [63].

Table 6.3: SQL `MATCH_RECOGNIZE` Pattern Examples [38]

Example 1: Detecting 'Dip' Patterns (Three Orders with Successively Lower Prices)
SQL Query

```
SELECT customer_id, start_date, end_date, bottom_price
FROM orders
MATCH_RECOGNIZE (
  PARTITION BY customer_id
  ORDER BY order_date
  MEASURES
    A.order_date AS start_date,
    C.order_date AS end_date,
    C.price      AS bottom_price
  PATTERN (A B C)
  DEFINE
    B AS B.price < A.price,
    C AS C.price < B.price
);
```

Trino Results:

```
customer_id | start_date | end_date   | bottom_price
------------+------------+------------+--------------
cust_1      | 2020-05-12 | 2020-05-16 | 50
(1 row)
```

Proposed `MATCH_RECOGNIZE` Results:

	customer_id	start_date	end_date	bottom_price
0	cust_1	2020-05-12	2020-05-16	50

Python UDF Results:

	customer_id	start_date	end_date	bottom_price
0	cust_1	2020-05-12	2020-05-16	50

6.6.1 Performance Results Using Real Dataset

The Amazon UK Products Dataset is a large e-commerce dataset containing information about products listed on Amazon's UK marketplace [16], commonly used for data analysis, machine learning, and benchmarking. It is provided as a single CSV file with roughly 2.2 million product records. Each record includes: `asin` (Amazon Standard Identification Number; unique product ID); `title` (product name); `imgUrl` (primary product image URL); `productURL` (canonical product page URL); `stars` (average customer rating, 1–5, floating-point); `reviews` (number of customer reviews, non-negative integer); `price` (listed price in GBP, numeric); `isBestSeller` (bestseller indicator, boolean); `boughtInLastMonth` (approximate purchases in the last 30 days, integer);

and `categoryName` (top-level category label, e.g. Electronics, Books, Clothing). This dataset is representative for price-trend analysis, brand/feature-popularity studies, pattern recognition, and recommender-system benchmarking.

The methodology involved testing 40 test cases across 5 SQL patterns and 8 input sizes (25 K, 35 K, 50 K, 75 K, 100 K, 150 K, 200 K, and 300 K rows). All tests completed successfully. The implementation achieved an average throughput of 9 444 rows/s, and the end-to-end runtime over all 40 test cases was approximately 550.34 s. The results indicate near-linear scaling over the evaluated range, with the expected performance differences between simple, alternation, quantified, optional, and nested patterns.

6.6.2 Pattern Definitions

Table 6.4: SQL Pattern Definitions and Detection Goals

Pattern Name	**SQL Pattern**	**Description & Goal**
simple_sequence	`A+ B+`	**Goal:** Detect transitions from unrated/poor to very poor products. **Use:** Identify sequences where no-rating/poor products (0–1.0 stars) are followed by very poor products (1.1–2.0 stars), useful for detecting problematic product sequences or data quality issues in listings.
alternation	`A (B|C)+ D`	**Goal:** Detect quality improvement patterns. **Use:** Find sequences starting with unrated/poor products, followed by mixed very poor/below-average products, ending with average-to-good products. Useful for analyzing quality improvement patterns in product browsing sequences or sorted lists.
quantified	`A{2,5} B* C+`	**Goal:** Detect constrained low-quality patterns. **Use:** Find sequences with 2–5 unrated/poor products, optionally followed by very poor products, then below-average products. Enforces minimum unrated product counts for identifying problematic product clusters.

Pattern Name	SQL Pattern	Description & Goal
optional_pattern	`A+ B? C*`	**Goal:** Flexible low-to-moderate quality transition detection. **Use:** Broad pattern matching starting with unrated/poor products with optional transitions through worse ratings. High recall for various low-quality sequence scenarios in e-commerce data.
complex_nested	`(A|B)+ (C{1,3} D*)+`	**Goal:** Complex quality improvement analysis. **Use:** Detect sequences of unrated/very poor products followed by groups of improving quality (below-average to good). Useful for multi-level quality grouping and advanced sorting pattern detection in product listings.

All patterns analyze product quality based on star ratings where A = No Rating/Poor (0–1.0 stars), B = Very Poor (1.1–2.0 stars), C = Below Average (2.1–3.0 stars), D = Average to Good (3.1–4.0 stars), E = Excellent (4.1–5.0 stars). Categories were created using equal-width binning on the `stars` column. Table 6.4 provides detailed specifications of all five SQL patterns used in the evaluation, including their formal syntax and intended detection goals. Each pattern represents a different complexity level and use case, ranging from simple sequential matching to complex nested structures with quantifiers and alternation operators.

6.6.3 Complexity Analysis

This subsection explains complexity in two complementary ways: *theoretical analysis* and *empirical analysis*. The theoretical analysis describes how the algorithm should grow as the input grows. The empirical analysis reports what was actually measured in the experiments. These two views are related, but they are not the same: theory predicts scaling, while experiments measure the current implementation on a specific machine and dataset.

Table 6.5: Theoretical and empirical views of complexity analysis

Theoretical Analysis	**Empirical Analysis**
Mathematical analysis.	Experimental measurement.
Independent of hardware and software environment.	Depends on the hardware, operating system, Python runtime, `pandas`, and memory behavior.
Uses asymptotic notation such as Big-O, Big-Θ, and Big-Ω.	Uses actual execution time, throughput, and memory usage.
Predicts scalability as input size grows.	Measures real-world performance for the evaluated benchmark.

Theoretical analysis. The current implementation can be viewed as four main steps:

1. parse the SQL query and extract the `MATCH_RECOGNIZE` clauses;
2. compile the row pattern into an internal automaton;
3. partition and order the input rows;
4. scan the ordered rows and build the output matches.

Let n be the number of input rows, p the number of partitions, n_i the number of rows in partition i, k the number of matches produced, and ℓ the average number of rows returned per match. Thus, $\sum_{i=1}^{p} n_i = n$. For the five benchmark patterns used in this chapter, the pattern size is fixed while the dataset size grows from 25 K to 300 K rows. The full execution path can be summarized as:

$$\text{time} \approx \text{compile pattern once} + \text{partition/order rows} + \text{scan rows} + \text{build output}.$$

The important point is the `ORDER BY` step. The current implementation uses `pandas` `sort_values` to order each partition before matching. If there are multiple partitions, the sorting cost is:

$$O\left(\sum_{i=1}^{p} n_i \log n_i\right).$$

This is at most $O(n \log n)$. If there is only one partition, it is the same as sorting the whole input. Therefore, the conservative theoretical cost for the current full pipeline is:

$$O(n \log n + n + k\ell).$$

Here, $O(n \log n)$ is the ordering cost, $O(n)$ is the scan over the ordered rows, and $O(k\ell)$ is the cost of building the output. If the data is already ordered before the matcher is called, the scan and output part alone is closer to $O(n + k\ell)$, but unordered daily data still requires sorting or another ordering strategy.

For daily workloads, where new rows often arrive out of order, the most practical future improvement is incremental ordering. Historical data can be stored in sorted form. When a new daily batch arrives, only the new batch is sorted, then it is merged with the already ordered history. This does not remove ordering completely, but it avoids sorting the full history again for every run.

The extra space used by the matcher comes mainly from partition/order structures, compiled pattern state, and output rows:

$$\text{extra space} \approx O(n + \text{pattern state} + k\ell).$$

The input DataFrame itself also uses memory proportional to the number of rows and columns.

Empirical analysis. The empirical analysis measures the implementation directly. In this chapter, it consists of 40 successful benchmark runs: five pattern families evaluated over eight dataset sizes from 25 K to 300 K rows.

The results show predictable behavior over the tested range. Average throughput is 9 444 rows/s, with values ranging from 5 570 to 12 798 rows/s. Throughput remains relatively stable as the input size increases, so the measured behavior is close to linear

for these workloads. This does not prove that the full algorithm is theoretically linear; it only shows how the current implementation behaves over the measured range.

The main performance difference comes from pattern complexity. The `simple_sequence` pattern is fastest because it has fewer transitions to check. The `quantified` and `complex_nested` patterns are slower because they contain more repetition, alternation, and nested structure. Memory use also remains bounded in the benchmark: Table 6.10 reports peak memory below 75 MB for all evaluated workloads. The memory values are not strictly increasing with dataset size because Python garbage collection, temporary objects, match density, and output shape affect the measured peak.

Table 6.6: Pattern Performance Summary (5 Patterns × 8 Dataset Sizes = 40 Tests)

Pattern	**Avg Throughput (rows/s)**	**Avg Time (s)**	**Test Cases**
simple_sequence	12,179	9.85	25K–300K
alternation	10,409	11.72	25K–300K
quantified	6,885	17.23	25K–300K
optional_pattern	11,451	10.48	25K–300K
complex_nested	6,297	19.51	25K–300K

Table 6.6 summarizes the performance characteristics of each pattern averaged across all eight dataset sizes. A clear inverse relationship exists between pattern complexity and throughput: simple patterns achieve nearly double the throughput of complex patterns. The `simple_sequence` pattern delivered the best throughput at 12 179 rows/s. The `complex_nested` pattern, while lower at 6 297 rows/s, successfully handles intricate nested structures. The `alternation` pattern maintains good throughput at 10 409 rows/s, and `optional_pattern` provides strong performance at 11 451 rows/s.

Table 6.7: Performance Summary by Dataset Size

Dataset Size (rows)	Avg Throughput (rows/s)	Avg Time (s)	Test Cases
25,000	10,150	2.65	5 patterns
35,000	9,619	3.92	5 patterns
50,000	9,928	5.41	5 patterns
75,000	9,551	8.43	5 patterns
100,000	9,306	11.47	5 patterns
150,000	8,942	18.21	5 patterns
200,000	9,202	23.59	5 patterns
300,000	8,856	36.39	5 patterns

Table 6.7 presents performance metrics organized by dataset size, averaging results across all five patterns. The table shows throughput stability across scales, with the mean throughput remaining between approximately 8 856 and 10 150 rows/s across the eight evaluated input sizes.

Table 6.8: Execution Time (seconds) by Pattern and Dataset Size

Pattern	Dataset Size (rows)							
	25K	35K	50K	75K	100K	150K	200K	300K
simple_sequence	1.96	2.82	3.91	6.07	8.34	13.07	16.27	26.35
alternation	2.13	3.26	4.58	7.15	10.02	15.29	19.91	31.44
quantified	3.44	5.23	7.08	10.73	14.59	22.27	29.13	45.37
optional_pattern	2.06	3.01	4.15	6.52	8.88	13.49	18.18	27.56
complex_nested	3.66	5.29	7.32	11.65	15.53	26.93	34.49	51.21

Table 6.8 presents a matrix of execution times for all pattern–size combinations. The table illustrates an approximately linear relationship between dataset size and execution time over the evaluated range. The `complex_nested` pattern consistently requires the longest execution time (51.21 s for 300 K rows), while `simple_sequence` achieves the fastest processing (1.96 s for 25 K rows).

6.6.4 Throughput by Pattern and Size

Table 6.9: Throughput (rows/s) by Pattern and Dataset Size

Pattern	Dataset Size (rows)							
	25K	35K	50K	75K	100K	150K	200K	300K
simple_sequence	12,727	12,403	12,798	12,348	11,997	11,477	12,293	11,383
alternation	11,762	10,741	10,906	10,482	9,977	9,809	10,047	9,541
quantified	7,275	6,690	7,058	6,988	6,854	6,735	6,866	6,612
optional_pattern	12,159	11,641	12,046	11,497	11,261	11,116	11,002	10,885

complex_nested	6,825	6,618	6,828	6,436	6,439	5,570	5,798	5,858

Throughput (rows/s) measures processing speed as dataset size divided by execution time; higher values indicate faster evaluation. Across the full 40-run benchmark, throughput ranged from 5,570 to 12,798 rows/s (mean 9,444 rows/s). Simple sequential patterns, which require fewer state transitions, typically achieve 12,000–13,000 rows/s, whereas complex nested patterns with multiple evaluation layers achieve about 6,000–7,000 rows/s. For a fixed pattern, throughput remains relatively constant as input size increases, supporting the near-linear scaling trend observed in Table 6.8.

Table 6.9 displays throughput measurements across all test scenarios. The table shows that most variation is driven by pattern complexity rather than input size: simple and optional patterns remain fastest, while quantified and nested patterns are consistently slower.

6.6.5 Memory Metrics

Memory Usage (MB) represents the average memory consumed during pattern matching operations. This includes memory for storing intermediate matching states, row buffers, and result sets. Memory usage varies based on both dataset size and pattern complexity. Several small-dataset runs remain below 3.20 MB, whereas runs with larger intermediate state or result structures can exceed 50 MB. The memory values shown have been corrected to absolute values, as some measurements showed negative artifacts due to garbage collection timing.

Peak Memory (MB) indicates the maximum memory consumption during pattern evaluation, typically occurring during result collection phases. Peak memory is approximately 1.3× the average memory usage, accounting for temporary allocations during pattern state transitions and match aggregation. Across the 40 memory-recorded cases in

Table 6.10, peak memory remains under 75 MB (maximum 74.51 MB at 300 K rows), indicating modest memory use for the evaluated workloads rather than a strictly monotonic relationship with dataset size. Notably, some larger tests consume less memory than smaller tests for different patterns, further validating the non-linear memory behavior.

Memory consumption does not correlate linearly with dataset size or match count, but rather depends on the complexity of intermediate pattern states and result-set characteristics. For example, at 100 K rows, `optional_pattern` finds 15,247 matches (8.4× more than `alternation`'s 1,828) but uses only 1.36 MB compared to `alternation`'s 30.12 MB, showing that match count alone does not determine memory usage. Most notably, `simple_sequence` at 75 K rows consumes 42.35 MB peak memory, while the same pattern at 100 K rows uses only 30.38 MB—a 28% reduction despite the larger dataset. This variability suggests memory usage is driven more by pattern state complexity, garbage collection timing, and internal representation than by output size or dataset scale. Despite these variations, the implementation shows bounded memory utilization for the benchmark range, with peak memory remaining within acceptable bounds (under 75 MB).

Table 6.10: Memory Consumption Metrics

Dataset Size (rows)	Pattern Complexity	Execution Time (ms)	Memory Usage (MB)	Peak Memory (MB)
25,000	simple_sequence	1,964	14.09	18.31
25,000	alternation	2,125	2.45	3.19
25,000	optional_pattern	2,056	6.38	8.29
25,000	quantified	3,436	1.27	1.66
25,000	complex_nested	3,663	0.46	0.60
35,000	simple_sequence	2,822	13.21	17.17
35,000	alternation	3,258	1.23	1.60
35,000	optional_pattern	3,006	7.68	9.98
35,000	quantified	5,231	3.68	4.78
35,000	complex_nested	5,288	10.95	14.24
50,000	simple_sequence	3,907	11.27	14.66
50,000	alternation	4,584	2.60	3.38

Table 6.10 (continued): Memory Consumption Metrics

Dataset Size (rows)	Pattern Complexity	Execution Time (ms)	Memory Usage (MB)	Peak Memory (MB)
50,000	optional_pattern	4,151	2.80	3.64
50,000	quantified	7,084	4.66	6.06
50,000	complex_nested	7,322	6.57	8.54
75,000	simple_sequence	6,074	32.57	42.35
75,000	alternation	7,155	8.71	11.32
75,000	optional_pattern	6,523	8.86	11.52
75,000	quantified	10,732	5.93	7.71
75,000	complex_nested	11,653	15.51	20.16
100,000	simple_sequence	8,335	23.37	30.38
100,000	alternation	10,023	30.12	39.15
100,000	optional_pattern	8,880	1.36	1.77
100,000	quantified	14,589	20.60	26.78
100,000	complex_nested	15,529	17.82	23.17
150,000	simple_sequence	13,069	31.71	41.22
150,000	alternation	15,291	44.97	58.46
150,000	optional_pattern	13,493	24.47	31.81
150,000	quantified	22,271	40.91	53.18
150,000	complex_nested	26,925	7.31	9.50
200,000	simple_sequence	16,268	53.64	69.74
200,000	alternation	19,906	44.44	57.77
200,000	optional_pattern	18,178	0.27	0.36
200,000	quantified	29,126	9.16	11.90
200,000	complex_nested	34,493	3.32	4.32
300,000	simple_sequence	26,353	52.98	68.87
300,000	alternation	31,443	57.32	74.51
300,000	optional_pattern	27,560	31.94	41.52
300,000	quantified	45,367	28.42	36.95
300,000	complex_nested	51,206	2.82	3.67

Table 6.10 presents detailed memory consumption metrics for all 40 benchmark scenarios, tracking both average and peak memory usage. The table shows bounded memory usage for the evaluated workloads, with peak memory remaining under 75 MB across all scenarios. The maximum peak memory of 74.51 MB occurs at 300 K rows

(`alternation`). The non-monotonic values across patterns and sizes illustrate that memory consumption depends on runtime factors including garbage collection timing, pattern state complexity, and the specific distribution of category transitions in the data.

6.6.6 Key Findings

Table 6.11: Overall benchmark test statistics

Metric	Value
Total Tests	40
Success Rate	100%
Total Execution Time	550.34 s
Average Execution Time	13.76 s
Average Throughput	9,444 rows/s
Min Throughput	5,570 rows/s
Max Throughput	12,798 rows/s

Table 6.11 presents the benchmark statistics aggregated across all 40 pattern–size runs. The system completed all evaluated patterns and dataset sizes successfully, with no failures or errors. Taken together, the results show that execution time increases predictably as dataset size grows and that the implementation handles patterns ranging from simple sequences to complex nested structures. This provides evidence of practical viability for medium-scale sequential pattern detection over e-commerce data.

6.7 CHAPTER SUMMARY

This chapter evaluated the proposed `MATCH_RECOGNIZE` implementation through correctness checks, feature-coverage tests, complexity analysis, performance measurements, memory analysis, and comparison with existing systems. The experiments used five pattern families across eight dataset sizes, producing 40 successful performance runs.

The results show that the implementation preserves the evaluated semantics while reducing developer effort compared with hand-written Pandas logic. The next chapter summarizes the thesis contributions, discusses limitations, and outlines future work.

CHAPTER 7
CONCLUSIONS AND FUTURE WORK

This chapter concludes the thesis by interpreting the main findings, stating the remaining limitations, and outlining future research directions. Section 7.1 discusses the practical meaning of the evaluation results. Section 7.3 summarizes the current limitations. Section 7.2 maps the chapter's claims to the evidence used to support them. Section 7.4 states the overall conclusion, and Section 7.5 presents future work.

7.1 DISCUSSION

The results show that a practical subset of SQL row-pattern recognition can be moved into `pandas` without abandoning the declarative structure of `MATCH_RECOGNIZE`. The correctness experiments compared selected reference queries against Trino 473, Oracle, and hand-written `pandas` baselines. For the reported reference cases, the proposed engine produced the same rows and measure values as the SQL baselines, including partition-local ordering, implicit `DEFINE` behavior, navigation with `PREV`, `MATCH_NUMBER()`, and measure materialization. These results do not imply broad equivalence with every feature of commercial SQL systems; instead, they provide evidence that the implemented subset preserves the evaluated semantics.

The comparison with idiomatic `pandas` is also important. Equivalent procedural implementations require explicit `groupby`, `shift`/`diff`, candidate filtering, index book-keeping, and manual handling of skip behavior. The SQL form expresses the same intent in one declarative statement. This reduces developer effort, makes the pattern definition easier to inspect, and supports reuse across notebook-based analysis and SQL workflows, provided that the query stays within the supported syntax and semantic subset.

The performance evaluation further supports the practical viability of the approach for medium-scale in-memory workloads. Chapter 6 reported 40 successful pattern–size

runs using five pattern families and eight dataset sizes from 25 K to 300 K rows. Across these runs, the implementation achieved an average throughput of 9 444 rows/s, with throughput remaining relatively stable as input size increased. The main variation came from pattern structure: simple and optional patterns were fastest, while quantified and nested patterns incurred additional transition and predicate-evaluation overhead. The memory measurements were also bounded for the evaluated range, with peak memory remaining below 75 MB across the 40 reported scenarios.

Architecturally, the system benefits from a separation between parsing, semantic validation, automata construction, matching, and result materialization. The implementation uses an ANTLR4 parser based on the Trino grammar, normalized internal representations, NFA/DFA construction, a `pandas`-oriented execution layer, and measure evaluation. This modular structure makes the implementation easier to validate and easier to extend. It also limits the effect of future changes: improving pattern compilation, adding a backend, or extending measure evaluation can be done without redesigning the entire engine.

The optimization layer contributes to this result but should be interpreted precisely. The implementation includes LRU-style pattern caching, bounded memory-management mechanisms, condition-cache pre-warming, and a parallel-aware scheduling abstraction for partition work. The current thesis evaluation, however, should be read as an evaluation of the implemented `pandas` execution path rather than as evidence for a distributed or broadly parallel SQL engine. This distinction is important because SQL engines such as Trino and Oracle provide mature optimizers, broader dialect coverage, and execution infrastructure that remain outside the scope of the present prototype.

The central finding is therefore scoped but meaningful: the thesis demonstrates that SQL row-pattern semantics can be compiled into a finite-automata-based engine and executed directly over DataFrames, while producing correct results for the evaluated reference scenarios and predictable performance over the measured input range.

7.2 VALIDATION OF CHAPTER CLAIMS

Table 7.1 summarizes the main claims made in this chapter, the evidence supporting each claim, and the boundary conditions under which the claim should be interpreted. This mapping is intended to keep the conclusion scientifically scoped: each claim is linked either to the correctness comparisons, the implementation description, or the measured benchmark results.

Table 7.1: Validation mapping between thesis claims, supporting evidence, and scope boundaries

Claim area	**Supporting evidence**	**Validated conclusion**	**Scope boundary**
Correctness	Selected Trino 473, Oracle, and hand-written `pandas` comparisons in Chapter 6.	The reported reference queries produce the same rows and measure values across the compared systems.	The evidence covers selected supported scenarios, not every SQL:2016 or vendor-specific case.
Feature support	Implemented parser, semantic model, NFA/DFA compilation, output modes, skip policies, navigation, subsets, anchors, exclusion blocks, and `PERMUTE` support described in Chapters 4 and 5.	The system supports a practical row-pattern subset suitable for DataFrame-centered analysis.	Unsupported syntax, uncommon edge cases, and cost-based optimizer behavior remain outside the thesis scope.
Performance	The 40-run Amazon UK Products benchmark over five pattern families and eight dataset sizes from 25 K to 300 K rows.	Execution time scales predictably over the measured range, with average throughput of 9 444 rows/s.	The benchmark does not establish performance for very large, distributed, or highly adversarial workloads.

Table 7.1 (continued): Validation mapping between thesis claims, supporting evidence, and scope boundaries

Claim area	Supporting evidence	Validated conclusion	Scope boundary
Memory use	Memory measurements for all 40 benchmark scenarios, including the maximum observed peak memory of 74.51 MB.	Memory use remains bounded for the evaluated workloads.	The measurements are empirical and do not define a formal upper bound for all possible patterns or datasets.
Developer effort	Comparison between declarative `MATCH_RECOGNIZE` queries and hand-written `pandas` baselines using grouping, shifting, and manual skip handling.	The SQL form reduces procedural code and improves readability for the evaluated pattern tasks.	The claim is based on code-level comparison, not a controlled user study.
Architecture	The modular parser–AST–automata–matcher–measure-evaluator pipeline described in Chapter 5.	The design separates concerns and creates extension points for future backends and optimizations.	The current implementation remains a `pandas`-centered prototype, not a mature distributed SQL optimizer.

The validation evidence also has three important threats to validity. First, *internal validity* depends on the correctness of the reference baselines and on careful normalization of case-only column differences between SQL and `pandas` outputs. Second, *construct validity* depends on whether the selected metrics capture the intended qualities: row-level equality captures output correctness for the tested cases, throughput captures execution speed, and line-count comparison captures only one aspect of developer effort. Third, *external validity* is limited by the use of one workstation, one main real-world dataset, and a bounded range of input sizes. These threats do not invalidate the reported results, but they define the conditions under which the results should be generalized.

7.3 LIMITATIONS

Several limitations remain and should be considered when interpreting the results.

Supported subset rather than exhaustive SQL coverage. The implementation targets a substantial but bounded subset of `MATCH_RECOGNIZE`. It covers core clauses such as `PARTITION BY`, `ORDER BY`, `PATTERN`, `DEFINE`, selected `MEASURES`, supported skip modes, output modes, navigation functions, subsets, exclusion blocks, anchors, and `PERMUTE`. It does not claim exhaustive SQL:2016 coverage or feature parity with Trino, Oracle, or other production SQL engines. Vendor-specific extensions and uncommon edge cases remain outside the validated scope.

Complex pattern and quantifier interactions. The automata-based design handles ordinary sequences, alternation, grouping, bounded repetitions, and common quantified patterns. However, deeply nested expressions with multiple greedy or unbounded quantifiers can increase the number of states and transitions substantially. Constructs such as `PERMUTE` are especially expensive because their direct semantics involve factorial expansion, so the implementation relies on complexity guards and conservative fallback behavior for large cases.

Measure and aggregate expression scope. The engine supports selected aggregate and scalar expressions used by the implemented measure evaluator, including commonly used functions such as `SUM`, `COUNT`, `MIN`, `MAX`, `AVG`, statistical aggregates, collection/string aggregates, conditional aggregates, and supported `RUNNING`/`FINAL` forms. Nevertheless, arbitrary user-defined aggregate functions and all possible SQL expression combinations remain outside the current scope. This is a natural limitation of an in-memory prototype that implements its own expression evaluator rather than reusing a general SQL execution engine.

Performance boundaries. The benchmark demonstrates predictable behavior from 25 K to 300 K rows on the evaluated workstation. Larger datasets, wider schemas,

more partitions, and highly ambiguous patterns may expose bottlenecks from Python execution, `pandas` group processing, row-context construction, and predicate evaluation. Native SQL engines and compiled columnar systems can still provide higher throughput for large-scale workloads.

Memory behavior. Memory usage remained bounded in the reported benchmark, but the measurements were non-monotonic across pattern families and dataset sizes. This behavior is consistent with garbage collection timing, intermediate state representation, partition grouping, and result-set materialization. Therefore, the reported memory results should be interpreted as empirical measurements for the tested workloads rather than as a formal upper bound for all possible patterns.

Optimizer integration. The engine operates independently of database query optimizers. It does not currently perform cost-based plan selection, predicate pushdown into external storage, join reordering, or distributed resource scheduling. As a result, some plan-level optimizations available in database systems are not available in the current implementation.

7.4 CONCLUSION

This thesis presented a SQL-in-`pandas` implementation for executing a supported subset of `MATCH_RECOGNIZE` over DataFrames. The work contributes a parsing and validation pipeline based on Trino SQL syntax, an internal representation of row-pattern queries, finite-automata-based pattern compilation, a `pandas`-based matcher, measure evaluation, and optimization support through caching and memory-aware execution mechanisms.

The evaluation provides three main conclusions. First, the selected correctness scenarios matched the reference outputs from Trino, Oracle, and hand-written `pandas` baselines. Second, the declarative SQL formulation substantially reduces the amount of procedural Python code needed to express sequential row-pattern analysis. Third,

the benchmark over the Amazon UK Products Dataset shows stable medium-scale performance across 40 pattern–size runs, with an average throughput of 9 444 rows/s and bounded memory use in the measured range.

The proposed approach is therefore best understood as a bridge between SQL row-pattern recognition and DataFrame-centered analytics. It does not replace mature SQL engines, but it makes a useful subset of their row-pattern functionality available inside Python workflows where data are already represented as `pandas` DataFrames.

7.5 FUTURE WORK

Future work can extend the implementation along five complementary directions.

Broader SQL and dialect coverage. The most direct extension is to increase coverage of SQL:2016 `MATCH_RECOGNIZE` semantics and selected vendor behavior. This includes additional navigation combinations, broader expression support, richer aggregate handling, more extensive `RUNNING`/`FINAL` semantics, and a larger conformance suite based on Trino, Oracle, and standard-derived examples.

Parallel and distributed execution. Independent partitions provide a natural unit of parallel work. Future versions can connect the existing parallel-aware scheduling abstraction to real concurrent execution, while preserving deterministic partition ordering and skip semantics. A further step is distributed execution over frameworks such as Dask or Spark, which would require boundary-aware state propagation for patterns whose matches may span partition chunks.

Backend abstraction and columnar acceleration. The current execution path is centered on `pandas`. A backend abstraction could target Polars, Apache Arrow, or other columnar engines. This would allow more predicate evaluation to run in compiled vectorized kernels and could reduce Python-level row iteration. Additional acceleration through Cython, Numba, or specialized compiled predicates is also a

promising direction, but it requires careful validation to preserve SQL-style null handling, navigation semantics, and deterministic output.

Optimization and diagnostics. The automata pipeline can be improved through stronger state pruning, guard pushdown, more precise selectivity estimates, and better handling of high-ambiguity quantified patterns. Diagnostic tooling is also important: explain plans, NFA/DFA visualizations, match traces, and per-pattern performance counters would make it easier to debug incorrect patterns and understand performance behavior.

Usability and integration. Future work can improve the user-facing workflow by adding clearer error messages, notebook-friendly visualizations, reusable test fixtures, and integration with existing DataFrame query interfaces. These additions would make the engine easier to adopt in exploratory data analysis and teaching settings.

7.6 CHAPTER SUMMARY

This chapter summarized the thesis findings and placed them in a scientifically scoped context. The implementation was shown to support a useful subset of `MATCH_RECOGNIZE` over `pandas`, with correctness evidence from selected reference queries and performance evidence from a 40-run benchmark over medium-scale DataFrames.

The chapter also identified the main limitations: bounded SQL coverage, complex-pattern growth, expression-evaluation scope, large-scale performance constraints, memory variability, and limited optimizer integration. The proposed future work targets these issues through broader conformance testing, parallel and distributed execution, backend abstraction, optimization improvements, and better developer tooling.

BIBLIOGRAPHY

[1] Dušan Petković. "Identifying Possible Financial Frauds using SQL Row Pattern Recognition." In: *International Journal of Computer Applications* 184.35 (Nov. 2022), pp. 31–34. ISSN: 0975-8887. DOI: 10.5120/ijca2022922446. URL: https://ijcaonline.org/archives/volume184/number35/32542-2022922446/.

[2] International Organization for Standardization. *ISO/IEC TR 19075-5:2016 Information technology – Database languages – SQL Technical Reports – Part 5: Row Pattern Recognition in SQL, Feature R010: Row pattern recognition in FROM clause*. https://www.iso.org/standard/65143.html. 2016.

[3] International Organization for Standardization and International Electrotechnical Commission. *Information technology — Guidance for the use of database language SQL — Part 5: Row pattern recognition*. https://www.iso.org/standard/78936.html. ISO/IEC 19075-5:2021, Edition 1. 2021.

[4] Oracle. *Pattern Recognition With MATCH_RECOGNIZE*. https://docs.oracle.com/en/middleware/fusion-middleware/osa/19.1/cqlreference/pattern-recognition-match_recognize.html. 2022.

[5] Kim Hansen. *Practical Oracle SQL: Mastering the Full Power of Oracle Database*. Apress Berkeley, CA, Jan. 2020. ISBN: 978-1-4842-5616-9. DOI: 10.1007/978-1-4842-5617-6.

[6] Trino. *MATCH_RECOGNIZE*. https://trino.io/docs/current/sql/match-recognize.html. 2022.

[7] Trino. *Row Pattern Recognition with MATCH_RECOGNIZE*. https://trino.io/blog/2021/05/19/row_pattern_matching.html. 2021.

[8] Snowflake Inc. *MATCH_RECOGNIZE Clause*. Snowflake Documentation. 2025. URL: https://docs.snowflake.com/en/sql-reference/constructs/match_recognize (visited on 06/18/2025).

[9] Apache Flink. *MATCH_RECOGNIZE*. https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/table/sql/queries/match_recognize/. 2026.

[10] Guyren Howe. *Any interest in adding MATCH_RECOGNIZE?* PostgreSQL General Mailing List, Message-ID: 7e495cb8-ca03-4e3e-aa7e-45bf75212da8@Spark. 2020. URL: https://www.postgresql.org/message-id/7e495cb8-ca03-4e3e-aa7e-45bf75212da8@Spark (visited on 06/18/2025).

[11] Tim Findling. *Bringing Row Pattern Recognition to DuckDB: Translating MATCH_RECOGNIZE to WITH RECURSIVE*. Bachelor's thesis, Eberhard Karls Universität Tübingen. 2025.

[12] Yu Cao et al. *Optimization of Analytic Window Functions*. 2012. arXiv: 1208.0086 [cs.DB]. URL: https://arxiv.org/abs/1208.0086.

[13] Wes McKinney. "Data Structures for Statistical Computing in Python." In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a.

[14] Jan Michels et al. "The new and improved SQL: 2016 standard." In: *ACM SIGMOD Record* 47.2 (2018), pp. 51–60.

[15] Dušan Petković. "Specification of Row Pattern Recognition in the SQL Standard and its Implementations." In: *Datenbank-Spektrum* 22.2 (July 2022), pp. 163–174. ISSN: 1610-1995. DOI: 10.1007/s13222-022-00404-3.

[16] asaniczka. *Amazon UK Products Dataset 2023*. Kaggle dataset. https://www.kaggle.com/datasets/asaniczka/amazon-uk-products-dataset-2023 (accessed 2025-07-16). 2023.

[17] Monier Lawande, Mohamed El-helw, and Ahmed Awad. "Row Pattern Recognition for Pandas: Bringing MATCH_RECOGNIZE to Python DataFrames." In: *2026 IEEE 23rd Mediterranean Electrotechnical Conference (MELECON)*. IEEE, 2026. DOI: 10.1109/MELECON64486.2026.11418885.

[18] Oracle Corporation. *Pattern Matching in Oracle Database 12c*. https://docs.oracle.com/database/121/DWHSG/pattern_match.htm. Accessed: 2024. 2013.

[19] ISO/IEC. *ISO/IEC 9075-2:2016 Information technology — Database languages — SQL — Part 2: Foundation (SQL/Foundation)*. International Standard. International Organization for Standardization, 2016.

[20] Felipe Hoffa. *Funnel Analytics with SQL: MATCH_RECOGNIZE() on Snowflake*. Medium, Towards Data Science. Mar. 2021. URL: https://medium.com/data-science/funnel-analytics-with-sql-match-recognize-on-snowflake-8bd576d9b7b1 (visited on 06/15/2025).

[21] Markus Winand. *What's new in SQL:2016*. Modern SQL: A lot has changed since SQL-92. June 2017. URL: https://modern-sql.com/blog/2017-06/whats-new-in-sql-2016 (visited on 06/15/2025).

[22] Markus Winand. *match_recognize — Regular Expressions Over Rows*. Modern SQL: A lot has changed since SQL-92. URL: https://modern-sql.com/feature/match_recognize (visited on 06/15/2025).

[23] Jim Melton and Andrew Eisenberg. “SQL:2016 Support for Row Pattern Recognition.” In: *SIGMOD Record* 46.2 (2017), pp. 39–48. DOI: 10.1145/3105906.3105914.

[24] Fred Zemke et al. *Pattern Matching in Sequences of Rows*. Technical Report. ANSI Standard Proposal, 2007.

[25] University of Tübingen, Database Systems Group. *Match_Recognize Research Project*. Database Systems Teaching Notes. URL: https://db.cs.uni-tuebingen.de/teaching/research-projects/match-recognize/ (visited on 07/07/2025).

[26] University of Tübingen, Database Systems Group. *Research Projects*. Database Systems Teaching Notes. URL: https://db.cs.uni-tuebingen.de/teaching/research-projects/ (visited on 07/07/2025).

[27] Oracle. *SQL Pattern Matching in Oracle Database*. https://docs.oracle.com/en/database/oracle/oracle-database/21/dwhsg/sql-pattern-matching-data-warehouses.html. Oracle Database 21c Data Warehousing Guide. 2021. (Visited on 06/15/2025).

[28] Šimun Šprem et al. “Building Advanced Web Applications Using Data Ingestion and Data Processing Tools.” In: *Electronics* 13.4 (2024), p. 709.

[29] Elena Georgiou. *Evaluating Apache Flink, Kafka Streams, and Spark Streaming for Scalable Machine Learning Workflows*. 2025.

[30] Maximilian Schule et al. “In-Database Machine Learning with SQL on GPUs.” In: *Proceedings of the 33rd International Conference on Scientific and Statistical Database Management*. SSDBM ’21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 25–36. ISBN: 9781450384131. DOI: 10.1145/3468791.3468840. URL: https://doi.org/10.1145/3468791.3468840.

[31] Nantia Makrynioti, Ruy Ley-Wild, and Vasilis Vassalos. “sql4ml: A declarative end-to-end workflow for machine learning.” In: *arXiv preprint arXiv:1907.12415* (2019). URL: https://api.semanticscholar.org/CorpusID:198968138.

[32] D. Naphade. “The Evolution and Modernization of Data Pipeline Architectures.” In: *European Journal of Computer Science and Information Technology* 13.6 (2025), pp. 42–53.

[33] Mark Raasveldt and Hannes Mühleisen. “DuckDB: an Embeddable Analytical Database.” In: *Proceedings of the 2019 International Conference on Management of Data*. Amsterdam, Netherlands: Association for Computing Machinery, 2019, pp. 1981–1984. DOI: 10.1145/3299869.3320212.

[34] Torben Ægidius Mogensen. *Introduction to Compiler Design*. 3rd. Springer Nature, 2024, pp. 105–113.

[35] Ji Lucas et al. "RheemStudio: Cross-Platform Data Analytics Made Easy." In: *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. 2018, pp. 1573–1576. DOI: 10.1109/ICDE.2018.00179.

[36] Angelo Mozzillo et al. *Evaluation of Dataframe Libraries for Data Preparation on a Single Machine*. 2025. DOI: 10.48786/EDBT.2025.27. URL: https://openproceedings.org/2025/conf/edbt/paper-96.pdf.

[37] Sanjay Chawla et al. "Building your Cross-Platform Application with RHEEM." In: *arXiv preprint arXiv:1805.11723* (2018). DOI: 10.48550/arXiv.1805.11723. URL: https://arxiv.org/abs/1805.11723.

[38] MonierAshraf. *Row_match_recognize*. GitHub repository. 2025. URL: https://github.com/MonierAshraf/Row_match_recognize/blob/5d58e8de6b1143e8c1d77cd70Examples%20Match_reocginze/Examples.ipynb.

[39] Yeisson Steven Tarazona Bernal. *ANTLR 4 grammar of the Swift 5 programming language*. 2021.

[40] Terence Parr. *The definitive ANTLR 4 reference*. The Pragmatic Bookshelf, 2013.

[41] Kai Salomaa and Sheng Yu. "NFA to DFA transformation for finite languages." In: *International Workshop on Implementing Automata*. Springer. 1996, pp. 149–158.

[42] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd. Boston, MA, USA: Addison-Wesley, 2006. ISBN: 978-0-321-48681-3.

[43] Andi Albrecht and contributors. *python-sqlparse Documentation*. https://sqlparse.readthedocs.io/en/latest/. Accessed: 25 June 2026. 2026.

[44] Toby Mao and contributors. *SQLGlot: Python SQL Parser and Transpiler*. https://github.com/tobymao/sqlglot. Accessed: 25 June 2026. 2026.

[45] Erik Klahn and contributors. *mo-sql-parsing: More SQL Parsing*. https://github.com/klahnakoski/mo-sql-parsing. Accessed: 25 June 2026. 2026.

[46] pganalyze and contributors. *libpg_query: PostgreSQL Parser as a Standalone Library*. https://github.com/pganalyze/libpg_query. Accessed: 25 June 2026. 2026.

[47] Lele Gaifax and contributors. *pglast: PostgreSQL Languages AST and Statements Prettifier*. https://pglast.readthedocs.io/en/latest/. Accessed: 25 June 2026. 2026.

[48] Christian Wagenknecht and Michael Hielscher. *Formale Sprachen, abstrakte Automaten und Compiler*. Springer, 2022.

[49] David M. Beazley. *PLY (Python Lex-Yacc)*. https://www.dabeaz.com/ply/. Accessed: 2024. 2023.

[50] Erez Shinan. *Lark – a modern parsing library for Python*. https://github.com/lark-parser/lark. Accessed: 2024. 2023.

[51] Terence Parr. *The Definitive ANTLR 4 Reference*. Lewisville, TX, USA: Pragmatic Bookshelf, 2013. ISBN: 978-1-934356-99-9.

[52] Terence Parr, Sam Harwell, and Kathleen Fisher. "Adaptive LL(*) Parsing: The Power of Dynamic Analysis." In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*. OOPSLA '14. ACM, 2014. DOI: 10.1145/2660193.2660202.

[53] Francisco Ortin et al. "An empirical evaluation of Lex/Yacc and ANTLR parser generation tools." In: *Computer Languages, Systems & Structures* 64 (2021).

[54] ANTLR. *ANTLR (ANother Tool for Language Recognition)*. https://www.antlr.org/. Accessed: Nov. 15, 2024. 2024.

[55] Terence Parr. *ANTLR4 Documentation*. https://github.com/antlr/antlr4/tree/dev/doc. Accessed: Nov. 15, 2024. 2024.

[56] ANTLR Team and Contributors. *Trino SQL Grammar for ANTLR*. https://github.com/antlr/grammars-v4/tree/master/sql/trino. 2022.

[57] Jeremy Jones. *Abstract Syntax Tree Implementation Idioms*. https://hillside.net/plop/plop2003/Papers/Jones-ImplementingASTs.pdf. 2003.

[58] Elisabeth Robson and Eric Freeman. *Entwurfsmuster von Kopf bis Fuß*. O'Reilly, 2022, pp. 614–615.

[59] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd. Boston, MA, USA: Addison-Wesley, 2006. ISBN: 978-0-321-45536-9.

[60] Ken Thompson. "Programming Techniques: Regular Expression Search Algorithm." In: *Communications of the ACM* 11.6 (1968), pp. 419–422. DOI: 10.1145/363347.363387.

[61] Hiroaki Yamamoto and Hiroshi Fujiwara. "A New Finite Automata Construction Using a Prefix and a Suffix of Regular Expressions." In: *IEICE Transactions on Information and Systems* (2021). ISSN: 1745-1361. DOI: 10.1587/transinf.2020FCP0010.

[62] Guangming Xing. “A simple way to construct NFA with fewer states and transitions.” In: *Proceedings of the 42nd Annual ACM Southeast Conference*. ACMSE ’04. New York, NY, USA: Association for Computing Machinery, 2004. ISBN: 1581138709. DOI: 10.1145/986537.986588. URL: https://doi.org/10.1145/986537.986588.

[63] Monier Ashraf. *Python UDF Example for Row Pattern Recognition*. GitHub repository. 2025. URL: https://github.com/MonierAshraf/Row_match_recognize/blob/master/Python_Examples/python_udf.ipynb.

[64] Monier Ashraf. *Oracle MATCH_RECOGNIZE — Stock Price Examples*. GitHub repository. 2025. URL: https://github.com/MonierAshraf/Row_match_recognize/blob/master/Examples%20Match_reocginze/Oracle_data_tables.sql.